Universidad Distrital Francisco José de Caldas
Ingeniería Electrónica · Análisis y Diseño de Microprocesadores

TALLER 5

Microprocesador de propósito específico: determinante por el algoritmo de Bareiss con salida BCD (algoritmo XS3)

Caja negra, cajas grises, pseudoalgoritmo, diagrama ASM y código VHDL

Herramienta: Xilinx ISE 14.7 (XST, ISim) · Tarjeta: Digilent Nexys A7-100T (XC7A100T-1CSG324C)

Nota para el equipo. Este documento explica un diseño de referencia. El enunciado advierte que los diseños parecidos entre equipos no se califican, así que revisen cada estado y cada bloque, entiendan por qué está ahí y adáptenlo antes de entregarlo. Las conclusiones del informe son responsabilidad del equipo.

Contenido

1. Resumen del diseño **3**
- Reglas de descripción . . . 3
- Requisitos del enunciado y de la clase . . . 3
- Archivos del repositorio . . . 4

2. Repaso del algoritmo de Bareiss **4**
- Ejemplo 3×3 . . . 4
- Pivote cero . . . 4

3. Caja negra **5**
- Caja negra de la tarjeta . . . 5

4. Cajas grises **6**
- Tope de la tarjeta . . . 6

5. La ALU **7**

6. Pseudoalgoritmo con el hardware disponible **8**

7. Diagrama ASM **10**
- Convenciones . . . 10
- Vista general . . . 10
- Hoja 1: espera y carga de la matriz (s0 a s3) . . . 11
- Hoja 2: pivote, búsqueda de fila, intercambio y banco (s4 a s9) . . . 12
- Hoja 3: productos, resta e inicio de la división (s10 a s15) . . . 13
- Hoja 4: división, escritura de a(i,j), avance y determinante (s16 a s19) . . . 14
- Hoja 5: algoritmo XS3 integrado y final (s20 a s26) . . . 15
- Tabla de estados . . . 16

8. Algoritmo XS3 dentro del ASM **17**
- Traza del XS3 para 362 . . . 17

9. División por restas sucesivas **18**

10. Señales del ASM **19**
- 10.1 Señales de control por estado (ASM → bloques) . . . 19
- 10.2 Señales de estado (bloques → ASM) . . . 19
- 10.3 Registros del ASM y anchos . . . 20

11. Interfaz con la tarjeta **21**
- Divisor de frecuencia . . . 21
- Antirrebote (uno solo) . . . 21
- Sincronizador y display . . . 21

12. Tiempo de ejecución **21**

13. Implementación en ISE 14.7 y la Nexys A7 **21**
- 13.1 Proyecto . . . 21
- 13.2 Cambiar la matriz (día de la evaluación) . . . 22
- 13.3 Operación en la tarjeta . . . 22
- 13.4 Matrices de prueba . . . 22
- 13.5 Pines (src/nexys_a7.ucf) . . . 23

14. Simulación **23**

15. Limitaciones y verificación pendiente **23**

Anexo A. Código VHDL **25**

1. Resumen del diseño

El procesador calcula el determinante de la matriz guardada en su ROM (de 2×2 a 5×5) con el algoritmo de Bareiss, guarda los pivotes en un banco de registros y entrega el determinante y el pivote que indica `dir_p` en BCD, convertidos con el algoritmo XS3. La ROM hace parte de la arquitectura: para calcular el determinante de otra matriz se cambia el archivo de la ROM y se vuelve a programar la FPGA.

Los bloques del camino de datos (ROM, RAM, multiplicador, ALU y banco de pivotes) están descritos aparte e interconectados en la entidad estructural `procesador`. Un solo ASM (`asm_bareiss`, 27 estados) maneja el direccionamiento y les distribuye las señales. La división por restas sucesivas y el algoritmo XS3 están integrados en ese ASM, y todas sus sumas y restas pasan por la **única ALU** del procesador, que sigue la tabla de funcionamiento del curso (M, s1, s0).

Reglas de descripción

Regla	Cómo se cumple
Solo std_logic, excepto la RAM	Todas las señales, puertos y constantes son std_logic o std_logic_vector. La RAM sigue el modelo del curso (`type mem is array … of std_logic_vector` y `conv_integer` en la línea de dirección); es el único lugar del diseño donde aparecen.
Sin integer, signed ni unsigned	No hay `integer`, `natural`, `signed`, `unsigned`, `boolean`, tipos enumerados ni genéricos. Los estados son constantes `std_logic_vector`; el `case st is / when s0 =>` se escribe igual que en los ejemplos de clase.
Sin variables ni descripciones tipo software	No hay `variable` ni lazos `for`/`while`. Las repeticiones del algoritmo las hace el ASM, un paso por ciclo de reloj, con contadores.
Una sola ALU	Entidad `alu` con la tabla del curso. La usan t1 − t2, la división (residuo − |p|), los valores absolutos (0 − x) y el +3 del XS3. No hay otro sumador/restador de datos en el diseño.
Un solo multiplicador	Entidad `multiplicador` (25 × 25 → 50 bits).
Sintetizable y con esquemático RTL claro	Un archivo por bloque y entidades estructurales (`procesador`, `det_bareiss`) que solo conectan bloques. La síntesis genérica con GHDL termina sin errores ni latches, con un solo sumador de 50 bits (dentro de la ALU) y un solo multiplicador.
Solo para simulación	Los testbench usan variables, integer, real, string y wait; no se sintetizan.

Requisitos del enunciado y de la clase

Requisito	Cómo lo cumple el diseño
Señal de inicio y señal de final	Entradas y salidas del procesador; `final = '1'` en el estado s26.
Matriz en ROM: 00h = n, luego los a(i,j) como en la Figura 1	Entidad `rom_matriz`: dirección `dira` de 5 bits y dato a de 6 bits en complemento a 2. Elementos por columnas: a11, a21, …, an1, a12, …
La ROM hace parte de la arquitectura	La ROM está dentro del procesador; no se ingresa por puertos.
Multiplicador, sumador/restador y división por restas sucesivas	Entidad `multiplicador`; la ALU hace de sumador/restador; la división son los estados s15 y s16 del ASM.
Memoria RAM para las iteraciones	Entidad `RAM_memory` (modelo del curso), 64 × 32 bits.
Pivotes en un banco de registros, visibles con los switches	Entidad `banco_pivotes`. dir_p = k (SW2..SW0) elige el pivote; el último pivote (k = n) es el determinante.
Determinante en los displays, en decimal	Salida `det` en BCD; con dir_p = 0 el display muestra det.
XS3 integrado al algoritmo de Bareiss, usando la ALU	Estados s20 a s25 del mismo ASM, un estado por paso del documento XS3; el +3 lo calcula la ALU (A + B).

Requisito	Cómo lo cumple el diseño
Un solo antirrebote	Entidad `antirrebote`, solo para el botón de inicio.

Archivos del repositorio

Archivo	Contenido
src/det_bareiss.vhd	Tope para la Nexys A7 (estructural).
src/procesador.vhd	Procesador: une el ASM con sus bloques (estructural).
src/asm_bareiss.vhd	ASM de Bareiss con la división y el XS3 integrados (s0 a s26).
src/alu.vhd	ALU con la tabla del curso (M, s1, s0).
src/multiplicador.vhd	Multiplicador con signo 25 × 25 → 50.
src/rom_matriz.vhd, src/ram.vhd, src/banco_pivotes.vhd	ROM con la matriz, RAM (modelo del curso), banco de pivotes.
src/divisor_frecuencia.vhd, src/antirrebote.vhd, src/sincronizador.vhd, src/display_8dig.vhd	Interfaz con la tarjeta.
src/nexys_a7.ucf	Pines y restricción de reloj (100 MHz).
sim/tb_alu.vhd	Testbench de las 8 operaciones de la ALU.
sim/tb_procesador.vhd, sim/pkg_prueba.vhd	Testbench autoverificado del procesador: sirve para cualquier ROM.
sim/tb_det_bareiss.vhd	Testbench de la tarjeta: botón con rebotes y lectura del display.
sim/modelo_referencia.py	Valores esperados y generador de src/rom_matriz.vhd.
sim/probar_matrices.py, sim/resultado_ghdl.txt	Prueba con las 8 matrices y sus resultados.
doc/figuras/*.png, doc/fuentes/*.py	Figuras de este documento y los scripts que las generan.

2. Repaso del algoritmo de Bareiss

Bareiss es una eliminación gaussiana sin fracciones. En el paso k, cada elemento de la submatriz que está debajo y a la derecha del pivote se actualiza con la fórmula (1) del enunciado:

```
a(i,j) = ( a(k,k)·a(i,j) − a(i,k)·a(k,j) ) / p        para k < i ≤ n,  k < j ≤ n
p = pivote del paso anterior (p = 1 en el primer paso)
```

Después del paso k, el pivote actual a(k,k) pasa a ser p. Tras n − 1 pasos, el determinante queda en a(n,n), que es el último pivote. La división siempre es exacta: por la identidad de Sylvester, después del paso k cada a(i,j) es el determinante de una submatriz de (k+1)×(k+1) de A, que es un entero.

Durante el paso k solo cambian los elementos con i > k y j > k. La fila k y la columna k no se modifican, por eso el cálculo se hace sobre la misma RAM, sin una segunda copia de la matriz.

Ejemplo 3×3

```
A = | 2 1 3 |      k = 1, p = 1:  a22 = (2·4 − 1·1)/1 = 7    a23 = (2·2 − 1·3)/1 = 1
    | 1 4 2 |                      a32 = (2·2 − 3·1)/1 = 1    a33 = (2·5 − 3·3)/1 = 1
    | 3 2 5 |      k = 2, p = 2:  a33 = (7·1 − 1·1)/2 = 3   →   det = 3
Banco de pivotes: pivote 1 = 2, pivote 2 = 7, pivote 3 = 3 = det
```

Pivote cero

Si a(k,k) = 0, el ASM busca la primera fila i > k con a(i,k) ≠ 0 (estado s5), intercambia las filas k e i en las columnas k a n (s6 a s8) e invierte el bit `signo`. Al final, det = −a(n,n) si hubo un número impar de intercambios, y ese valor con su signo es el que se guarda como último pivote, para que el pivote n siempre sea igual al determinante. Si no hay fila con a(i,k) ≠ 0, det = 0.

3. Caja negra

La caja negra es la del tablero de clase: entradas clk, rst, inicio y dir_p; salidas det, pivote y final. La ROM, la RAM, el multiplicador y la ALU están adentro. Es la entidad `procesador`.

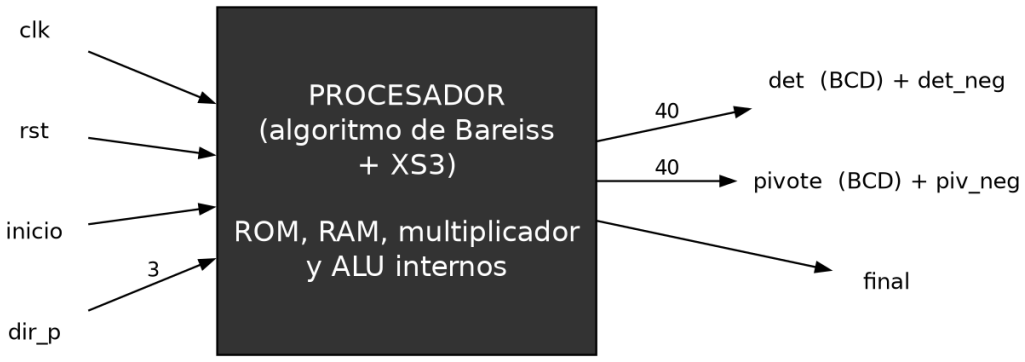


Figura 1. Caja negra del procesador.

Puerto	Dir.	Bits	Función
clk	in	1	Reloj (100 MHz en la tarjeta)
rst	in	1	Reinicio asíncrono, activo en alto
inicio	in	1	Comienza el cálculo (pulso de un ciclo; se lee en s0 y s26)
dir_p	in	3	Dirección del banco de pivotes: 1 a n = pivote k (el n es el det); 0, 6 y 7 leen 0
det, det_neg	out	40, 1	|det| en BCD (10 dígitos) y su signo
pivote, piv_neg	out	40, 1	|pivote(dir_p)| en BCD y su signo
final	out	1	'1' cuando det y pivote están listos (estado s26)

Caja negra de la tarjeta

La entidad `det_bareiss` conecta el procesador con la Nexys A7:

Puerto	Pin	Función
clk	E3	Reloj de 100 MHz
rst	BTNU (M18)	Reinicio de todo el diseño
inicio	BTNC (N17)	Comienza el cálculo (pasa por el antirrebote)
dir_p(2..0)	SW2..SW0	0: el display muestra det; 1 a 5: muestra el pivote k
final	LD0	Fin del cálculo
desborde	LD1	El valor mostrado tiene más de 7 dígitos
latido	LD15	Onda de 1 s del divisor de frecuencia (indica que el diseño está corriendo)
an, seg, dp	AN7..AN0, CA..CG, DP	Display: AN7 = signo, AN6..AN0 = 7 dígitos

4. Cajas grises

Antes del ASM se define el hardware: qué bloques hay, cómo se conectan y qué señales intercambian con la unidad de control. La caja gris del procesador muestra el ASM y los bloques del camino de datos con todas sus señales.

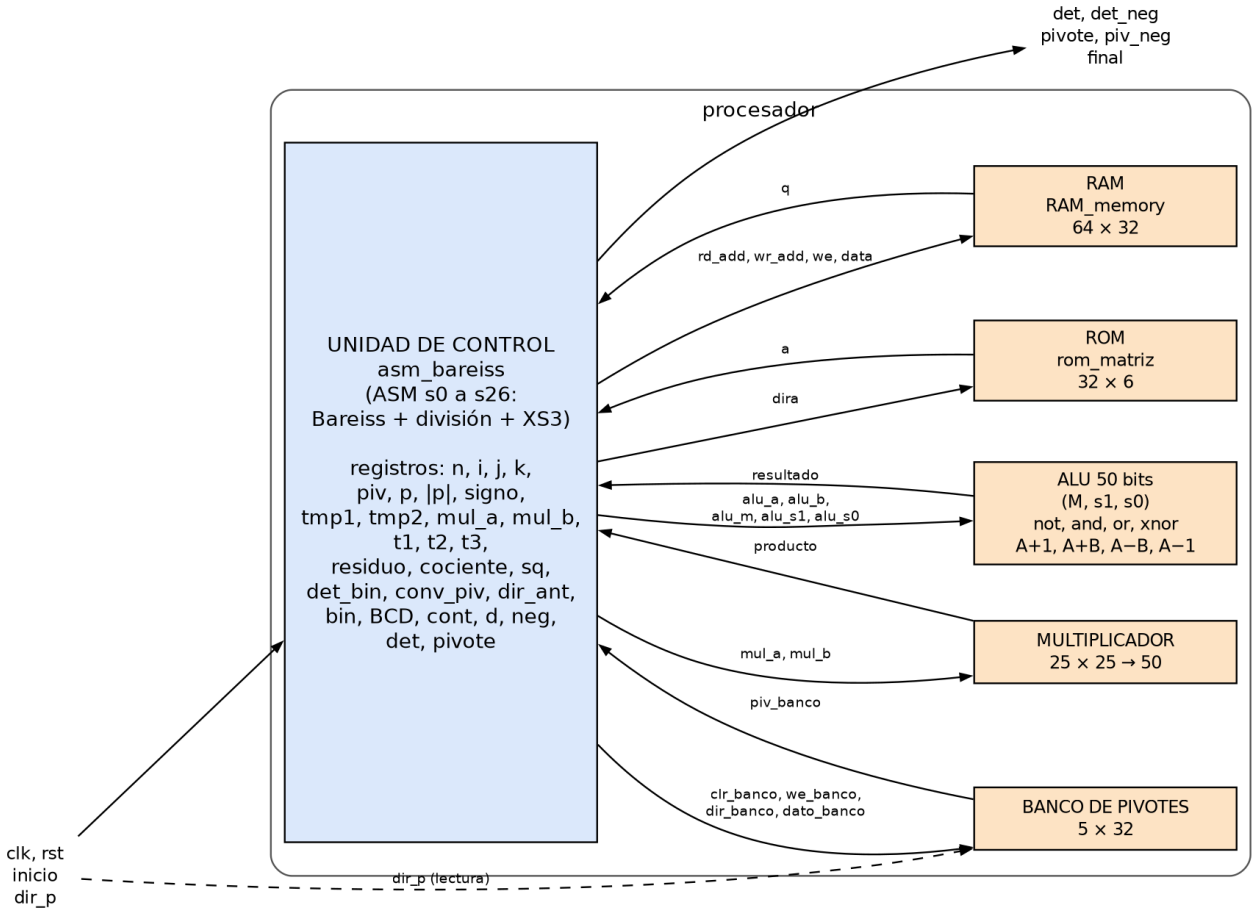


Figura 2. Caja gris del procesador: unidad de control (ASM) y bloques del camino de datos.

Bloque	Entradas desde el ASM	Salidas hacia el ASM	Uso
ROM (rom_matriz)	dira (5)	a (6)	Leer n y los elementos (s1, s2)
RAM (RAM_memory)	rd_add (6), wr_add (6), we, data (32)	q (32)	Matriz de trabajo; dirección fila & columna
multiplicador	mul_a (25), mul_b (25)	producto (50)	a(k,k)·a(i,j) y a(i,k)·a(k,j)
alu	alu_a (50), alu_b (50), alu_m, alu_s1, alu_s0	resultado (50)	t1 − t2, división, valores absolutos y +3 del XS3
banco_pivotes	clr_banco, we_banco, dir_banco (3), dato_banco (32)	piv_banco (32)	Guardar los pivotes; se lee con dir_p

Tope de la tarjeta

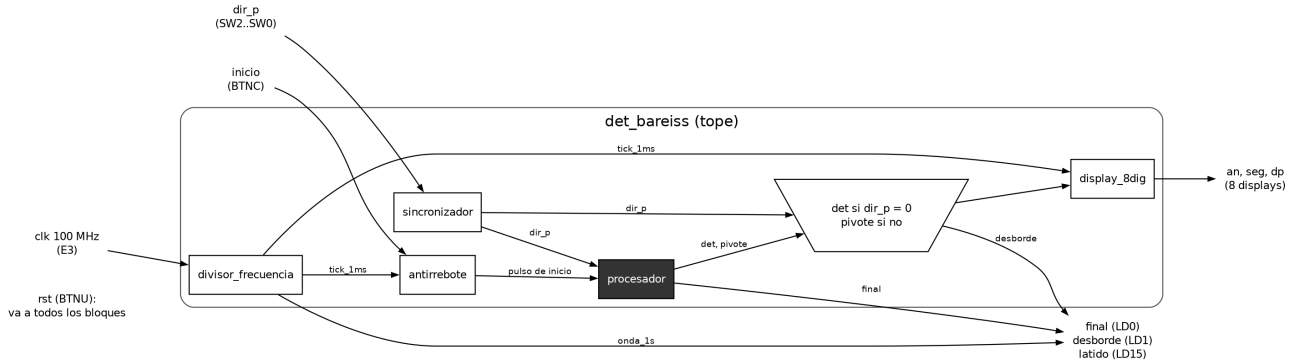


Figura 3. Bloques del tope det_bareiss.

5. La ALU

La entidad `alu` es la unidad aritmético-lógica del curso. Las líneas M, s1 y s0 eligen la operación:

M	s1	s0	Operación	Uso en este procesador
0	0	0	not B	—
0	0	1	A and B	—
0	1	0	A or B	—
0	1	1	A xnor B	—
1	0	0	A + 1	—
1	0	1	A + B	s23: dígito BCD + 3 (paso 3 del XS3)
1	1	0	A − B	s14: t1 − t2; s16: residuo − |p|; s15, s17, s18, s19, s20: valores absolutos 0 − x
1	1	1	A − 1	—

La ALU implementa las 8 operaciones aunque el algoritmo solo use A + B y A − B. Las cuatro operaciones aritméticas comparten un solo sumador `S = A + Bx + cin`; lo único que cambia es el segundo operando y el acarreo de entrada:

Operación	Bx	cin	Por qué
A + 1	0	1	A + 0 + 1
A + B	B	0	Suma directa
A − B	not B	1	Complemento a 2: −B = not B + 1
A − 1	1…1	0	1…1 es −1 en complemento a 2

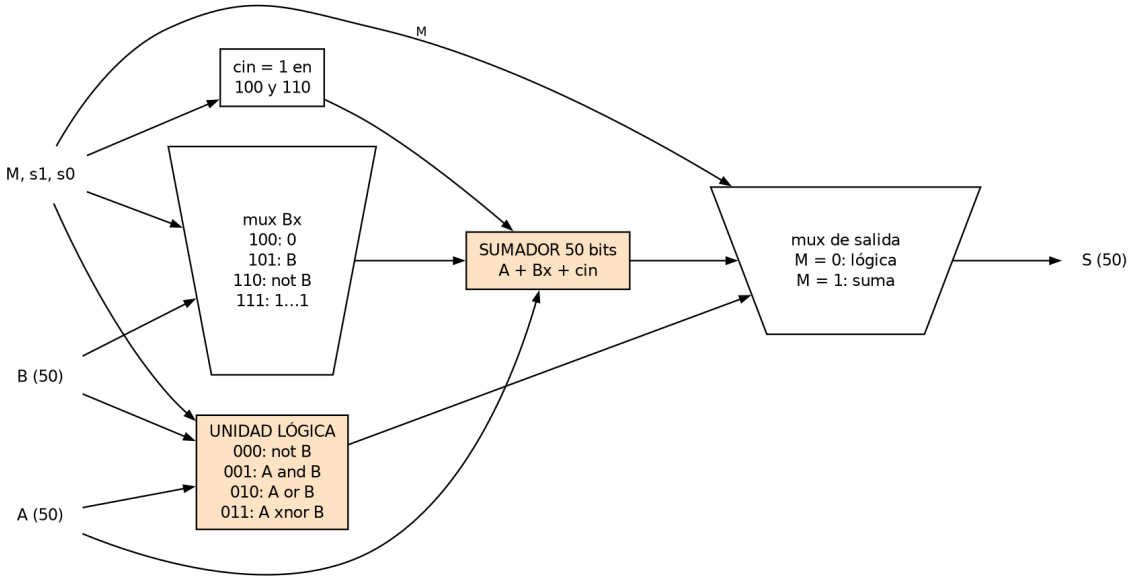


Figura 4. Estructura interna de la ALU.

La ALU es de 50 bits porque la operación más ancha es t1 − t2 (productos de 50 bits). El ASM elige los operandos con dos multiplexores según el estado (sección 10.1) y fija `M s1 s0 = 101` en s23 y `110` en los demás estados. La prueba sim/tb_alu.vhd verifica las 8 operaciones.

6. Pseudoalgoritmo con el hardware disponible

Cada línea indica el estado del ASM que la ejecuta. Las operaciones del mismo estado ocurren en el mismo ciclo de reloj. «RAM(f,c)» es la palabra de la RAM en la dirección fila f, columna c.

```
s0 :  final = 0;  esperar inicio = 1
s1 :  n = ROM(00h)
s2 :  para j = 1..n, i = 1..n:  RAM(i,j) = ROM(dira);  dira = dira + 1     (por columnas)
s3 :  k = 1;  p = 1;  |p| = 1;  signo = 0;  conv_piv = 0;  banco = 0
mientras k <= n-1:
  s4 :  piv = RAM(k,k)
        si piv = 0:
  s5 :     buscar i = k+1..n con RAM(i,k) /= 0;  si no existe: det_bin = 0, ir a XS3
  s6-s8:   intercambiar filas k e i (columnas k..n);  signo = not signo;  volver a s4
  s9 :  banco(k) = piv
        para i = k+1..n, j = k+1..n:
  s10-s13:  t1 = piv · RAM(i,j);   t2 = RAM(i,k) · RAM(k,j)      (multiplicador)
  s14:      t3 = t1 - t2                                        (ALU)
  s15:      residuo = |t3|;  cociente = 0                       (ALU para 0 - t3)
  s16:      mientras residuo - |p| >= 0:                        (ALU)
               residuo = residuo - |p|;  cociente = cociente + 1
  s17:      RAM(i,j) = +cociente o -cociente  (signo(t3) xor signo(p))
  s18:  p = piv;  |p| = |piv|;  k = k + 1
s19:  det_bin = RAM(n,n) o -RAM(n,n) (según signo);  banco(n) = det_bin
XS3 (dos veces: conv_piv = 0 para det, conv_piv = 1 para el pivote):
s20:  val = det_bin o banco(dir_p);  bin = |val|;  neg = signo(val)
s21:  PASO 1:  BCD = 0;  cont = 0
repetir:
  s22:  PASO 2:  BCD(0) = bin(31);   si cont = 31 salir   (PASO 5)
  s23:  PASO 3:  para d = 0..9:  si BCD(d) > 4 entonces BCD(d) = BCD(d) + 3   (ALU)
  s24:  PASO 4:  BCD = BCD << 1;  bin = bin << 1;  cont = cont + 1
s25:  si conv_piv = 0:  det = BCD;  conv_piv = 1;  repetir XS3
      si no:            pivote = BCD
s26:  final = 1;  si inicio = 1 ir a s1;  si cambia dir_p ir a s20
```

Los «para» y «mientras» no son lazos del VHDL: cada uno es un lazo del ASM que vuelve a un estado mientras un contador no llegue a su límite.

7. Diagrama ASM

Convenciones

Elemento	Significado
Caja de estado (rectángulo azul)	Operaciones de todo el estado (salidas tipo Moore).
Caja de decisión (rombo)	Pregunta sobre una señal de estado. La salida 1 es la condición verdadera.
Caja de salida condicional (rectángulo redondeado naranja)	Operación que solo ocurre si se toma ese camino (tipo Mealy), como en el ejemplo «ASM Mealy» de clase.
Bloque ASM (recuadro punteado)	Caja de estado más sus decisiones y salidas condicionales: dura un ciclo de reloj.
Conector (círculo verde)	Une hojas del diagrama: A → s1, B → s4, C → s10, D → s20, E → s16.
x ← expresión	Carga de un registro; se hace efectiva en el flanco de subida al final del ciclo.
x = expresión	Señal combinacional, por ejemplo final = 1.
(ALU: A − B)	La operación se hace en la ALU en ese estado.
RAM(f,c) ← dato	Escritura en la RAM (we = 1) en la dirección fila f, columna c.

Todas las decisiones de un bloque usan los valores que tienen los registros al comenzar el ciclo. Cada combinación de estado y decisiones lleva a un solo estado siguiente, y todo camino de decisiones termina en un estado.

Vista general

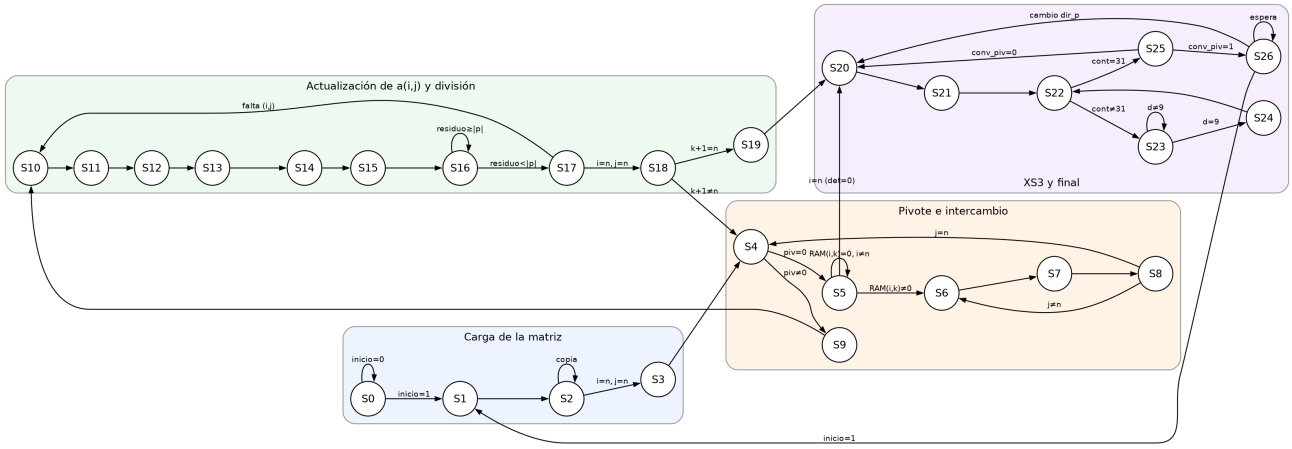


Figura 5. Diagrama de estados del ASM (27 estados).

Hoja 1: espera y carga de la matriz (s0 a s3)

s2 recorre la ROM con el contador dira y los contadores i y j. La dirección de la RAM es fila & columna (3 + 3 bits), así que no hace falta multiplicar para ubicar a(i,j).

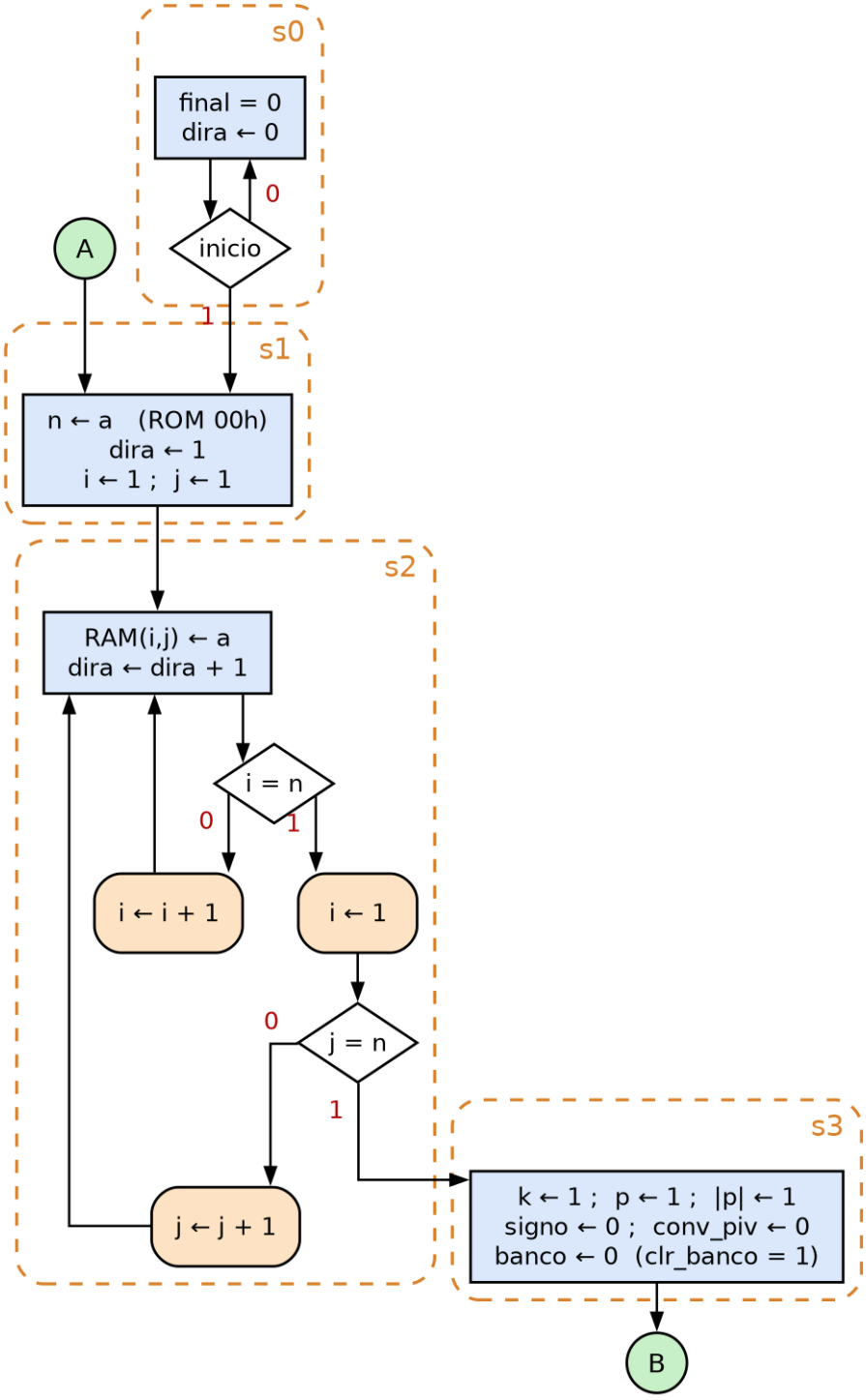


Figura 6. Hoja 1: espera y carga de la matriz (s0 a s3).

Hoja 2: pivote, búsqueda de fila, intercambio y banco (s4 a s9)

La RAM tiene una dirección de lectura y otra de escritura. En s7 se lee RAM(i,j) y en el mismo flanco se escribe tmp1 en esa posición; por eso el intercambio usa tres estados por columna.

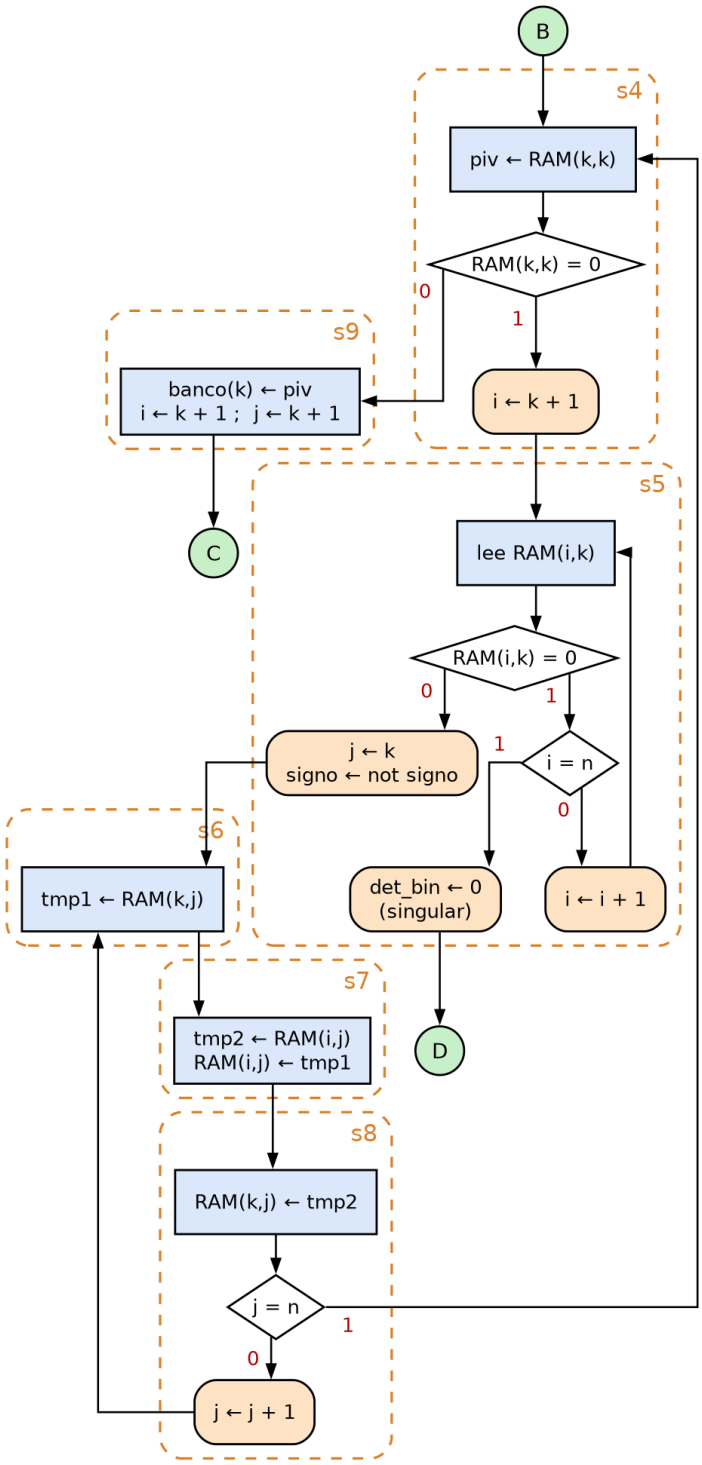


Figura 7. Hoja 2: pivote, búsqueda de fila, intercambio y banco (s4 a s9).

Hoja 3: productos, resta e inicio de la división (s10 a s15)

Con un solo multiplicador, los dos productos se hacen en estados distintos. La ALU calcula t1 − t2 en s14 y 0 − t3 en s15, para que el residuo empiece con |t3|.

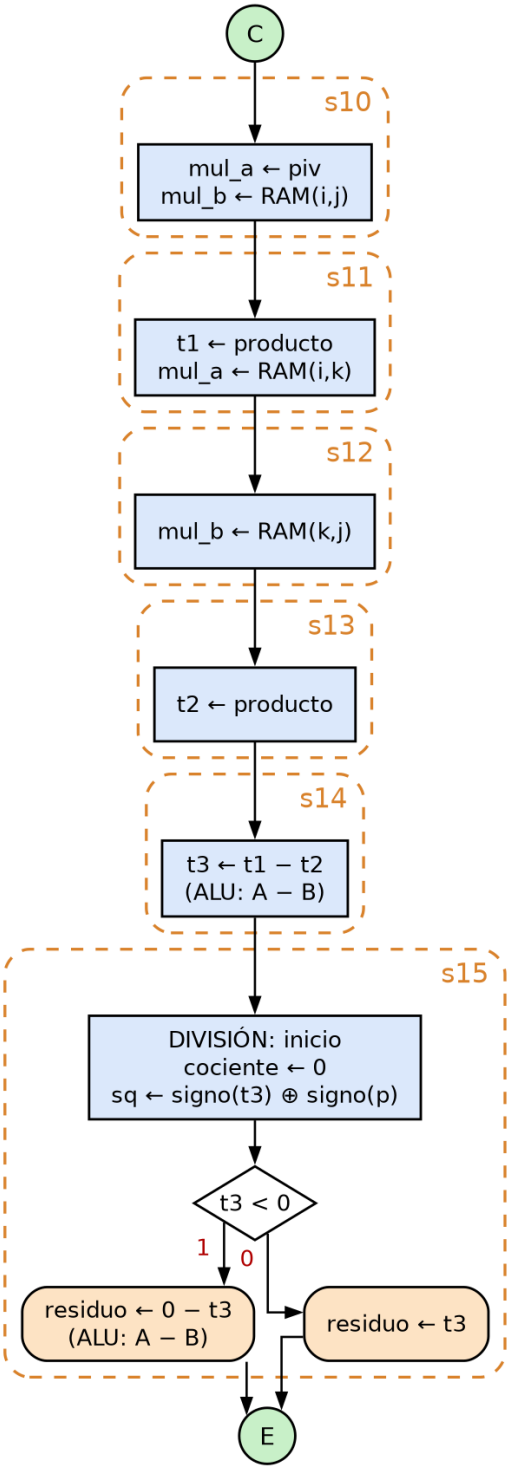


Figura 8. Hoja 3: productos, resta e inicio de la división (s10 a s15).

Hoja 4: división, escritura de a(i,j), avance y determinante (s16 a s19)

s16 es el lazo de la división por restas sucesivas: en cada ciclo la ALU calcula residuo − |p|; si no es negativo se guarda y el cociente aumenta. En s17 la ALU calcula 0 − cociente para escribir el valor con signo. s19 guarda el determinante como último pivote.

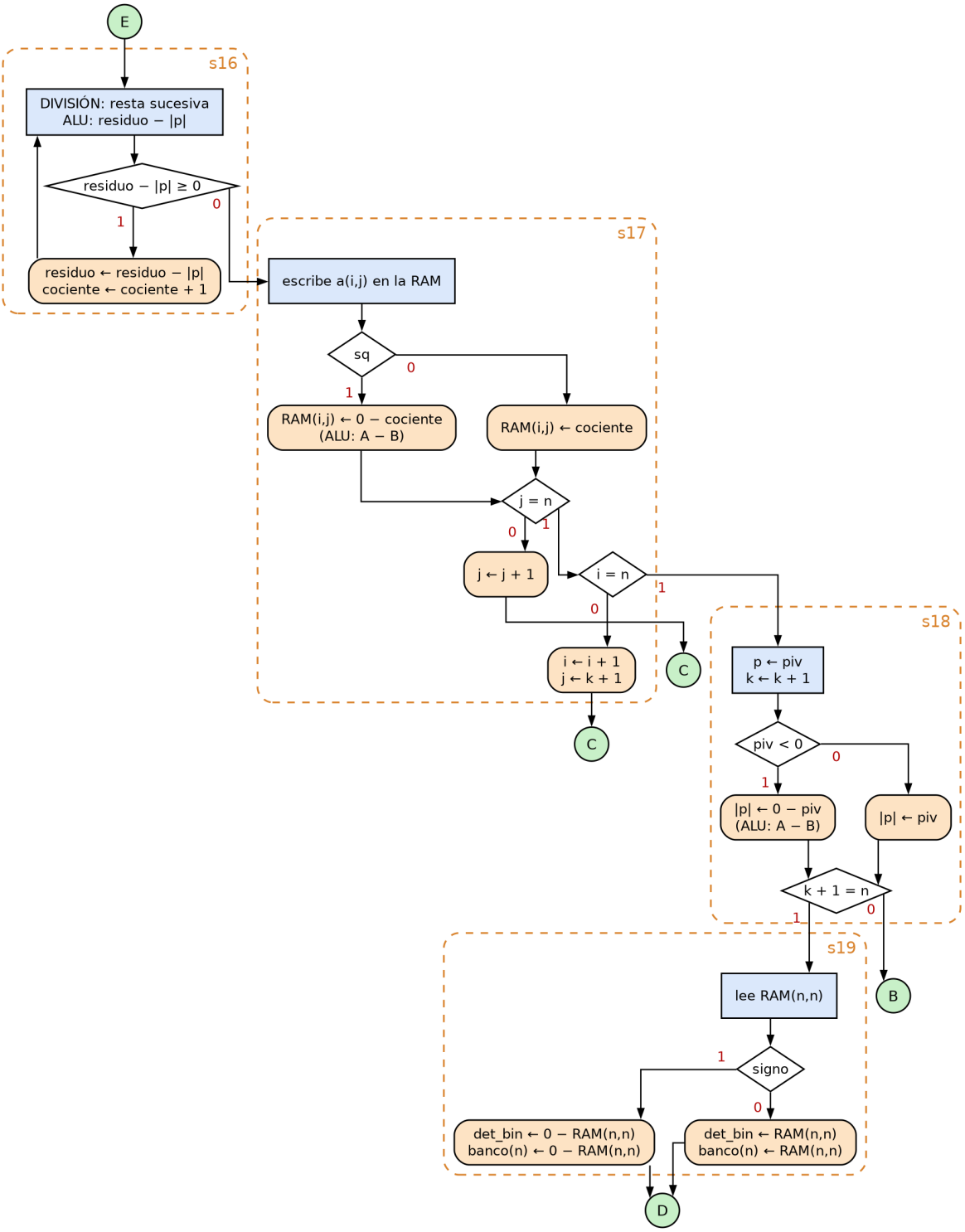


Figura 9. Hoja 4: división, escritura de a(i,j), avance y determinante (s16 a s19).

Hoja 5: algoritmo XS3 integrado y final (s20 a s26)

El XS3 convierte primero |det| y luego |pivote(dir_p)|, con un estado por paso del documento. En s23 la ALU calcula dígito + 3. En s26 el ASM espera con final = 1; si cambia dir_p vuelve a s20 y convierte el nuevo pivote.

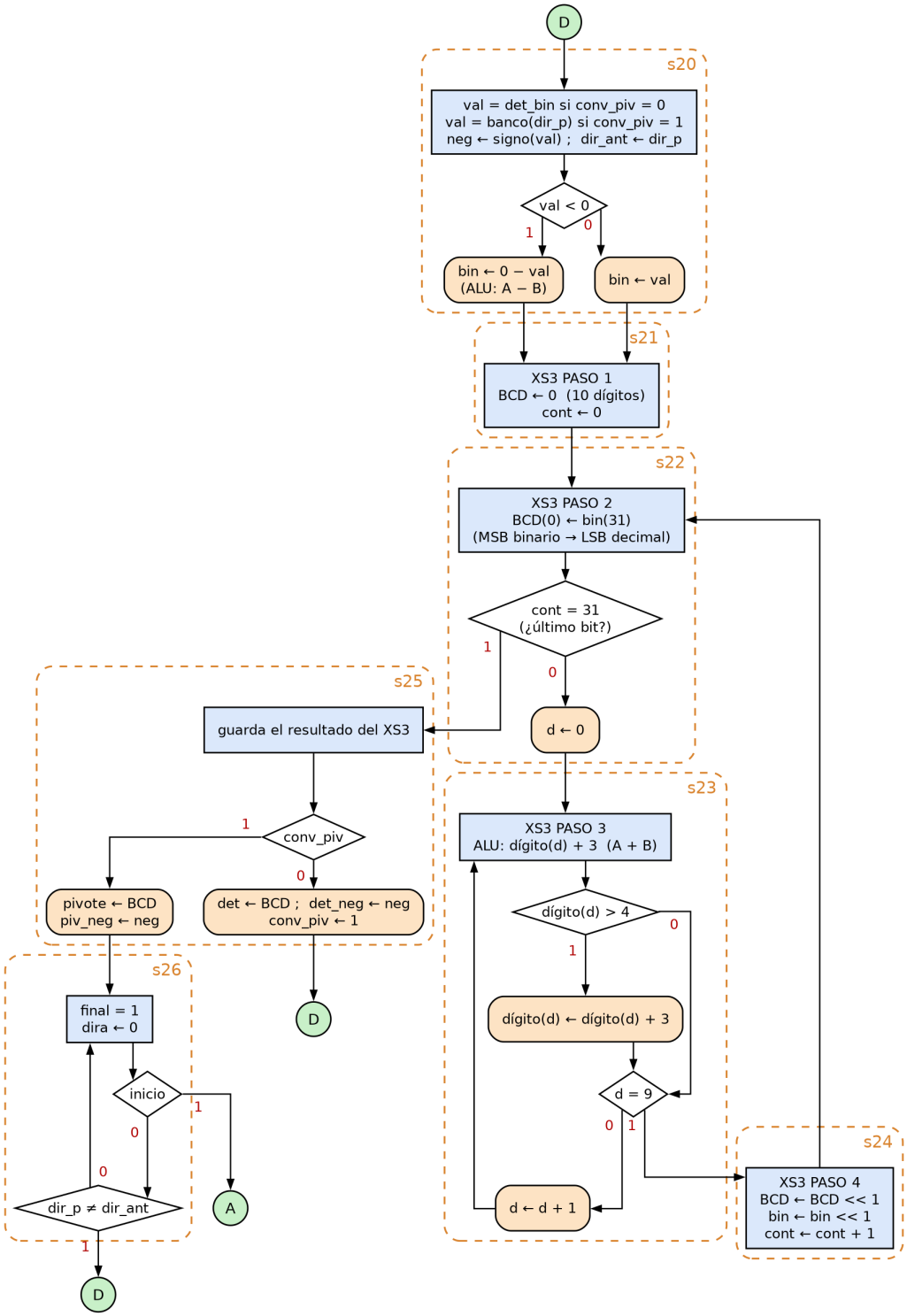


Figura 10. Hoja 5: algoritmo XS3 integrado y final (s20 a s26).

Tabla de estados

Resumen de cada bloque ASM, en el orden del archivo src/asm_bareiss.vhd.

Estado	Operaciones	Siguiente estado
s0	final = 0; dira ← 0	inicio = 1: s1. Si no: s0.
s1	n ← a (ROM 00h); dira ← 1; i ← 1; j ← 1	s2
s2	RAM(i,j) ← a (con extensión de signo); dira ← dira + 1	i ≠ n: i ← i + 1, s2. i = n y j ≠ n: i ← 1, j ← j + 1, s2. i = n y j = n: s3.
s3	k ← 1; p ← 1; |p| ← 1; signo ← 0; conv_piv ← 0; banco ← 0 (clr_banco)	s4
s4	piv ← RAM(k,k)	RAM(k,k) = 0: i ← k + 1, s5. Si no: s9.
s5	Lee RAM(i,k)	RAM(i,k) ≠ 0: j ← k, signo ← not signo, s6. RAM(i,k) = 0 e i = n: det_bin ← 0, s20. Si no: i ← i + 1, s5.
s6	tmp1 ← RAM(k,j)	s7
s7	tmp2 ← RAM(i,j); RAM(i,j) ← tmp1	s8
s8	RAM(k,j) ← tmp2	j = n: s4. Si no: j ← j + 1, s6.
s9	banco(k) ← piv; i ← k + 1; j ← k + 1	s10
s10	mul_a ← piv; mul_b ← RAM(i,j)	s11
s11	t1 ← producto; mul_a ← RAM(i,k)	s12
s12	mul_b ← RAM(k,j)	s13
s13	t2 ← producto	s14
s14	t3 ← t1 − t2 (ALU)	s15
s15	cociente ← 0; sq ← signo(t3) xor signo(p); residuo ← |t3| (ALU: 0 − t3 si t3 < 0)	s16
s16	ALU: residuo − |p|	residuo − |p| ≥ 0: residuo ← residuo − |p|, cociente ← cociente + 1, s16. Si no: s17.
s17	RAM(i,j) ← cociente, o 0 − cociente (ALU) si sq = 1	j ≠ n: j ← j + 1, s10. j = n e i ≠ n: i ← i + 1, j ← k + 1, s10. i = n y j = n: s18.
s18	p ← piv; |p| ← |piv| (ALU: 0 − piv si piv < 0); k ← k + 1	k + 1 = n: s19. Si no: s4.
s19	det_bin ← ±RAM(n,n); banco(n) ← ±RAM(n,n) (ALU: 0 − RAM(n,n) si signo = 1)	s20
s20	val = det_bin si conv_piv = 0, banco(dir_p) si conv_piv = 1; neg ← signo(val); dir_ant ← dir_p; bin ← |val| (ALU: 0 − val si val < 0)	s21
s21	XS3 paso 1: BCD ← 0; cont ← 0	s22
s22	XS3 paso 2: BCD(0) ← bin(31)	cont = 31: s25 (paso 5 cumplido). Si no: d ← 0, s23.
s23	XS3 paso 3: ALU dígito(d) + 3; si dígito(d) > 4, dígito(d) ← dígito(d) + 3	d ≠ 9: d ← d + 1, s23. d = 9: s24.
s24	XS3 paso 4: BCD ← BCD << 1; bin ← bin << 1; cont ← cont + 1	s22
s25	Guarda el resultado del XS3	conv_piv = 0: det ← BCD, det_neg ← neg, conv_piv ← 1, s20. conv_piv = 1: pivote ← BCD, piv_neg ← neg, s26.
s26	final = 1; dira ← 0	inicio = 1: s1. dir_p ≠ dir_ant: s20. Si no: s26.

8. Algoritmo XS3 dentro del ASM

El XS3 no es un bloque externo: sus pasos son estados del mismo ASM de Bareiss y su suma (+3) se hace en la ALU del procesador. Sus registros (binario, decimal y contadores) están en el ASM. Cada paso del documento XS3 tiene su estado:

Paso del documento XS3	Estado	Operación
1. Inicializar el registro decimal en ceros	s21	BCD ← 0 (40 bits); cont ← 0
2. Colocar el MSB del registro binario en el LSB del registro decimal	s22	BCD(0) ← bin(31)
3. Revisar cada dígito: si es ≥ 5 (> 0100), sumarle 3 (0011)	s23	Un dígito por ciclo (d = 0 a 9): un multiplexor entrega el dígito d, la ALU calcula dígito + 3 (A + B) y, si el dígito es > 4, se reemplaza por el resultado.
4. Correr a la izquierda y descartar el bit que sale; repetir desde el paso 2	s24	BCD ← BCD(38..0) & '0'; bin ← bin(30..0) & '0'; cont ← cont + 1; vuelve a s22
5. Realizar únicamente el paso 2	s22 con cont = 31	En la pasada 32 por s22 entra el último bit y el ASM va a s25.

Por qué se suma 3: si un dígito vale 5 o más, al correrlo a la izquierda quedaría en 10 o más, que no es un dígito BCD válido. Sumar 3 antes del corrimiento equivale a sumar 6 después, que es la corrección BCD, y hace que el exceso pase como acarreo al dígito siguiente. El registro binario es de 32 bits, así que el decimal necesita 10 dígitos ($2^{32} - 1 = 4\,294\,967\,295$) y hay 32 pasadas por el paso 2. El signo no entra al XS3: en s20 se carga |valor| en bin y el signo en neg.

Traza del XS3 para 362

$362 = 1\,0110\,1010_2$ tiene 9 bits significativos. En las pasadas cont = 0 a 22 solo entran ceros, por eso la tabla empieza en cont = 23. «bin(31..23)» son los 9 bits de arriba del registro binario; el de la izquierda, bin(31), es el que copia el paso 2. El registro decimal se muestra con sus tres dígitos bajos.

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
23	s22 (2)	0000 0000 0001	101101010	
	s23 (3)	0000 0000 0001	101101010	ningún dígito > 4
	s24 (4)	0000 0000 0010	011010100	corrimiento a la izquierda
24	s22 (2)	0000 0000 0010	011010100	
	s23 (3)	0000 0000 0010	011010100	ningún dígito > 4
	s24 (4)	0000 0000 0100	110101000	corrimiento a la izquierda
25	s22 (2)	0000 0000 0101	110101000	
	s23 (3)	0000 0000 1000	110101000	BCD(0) = 0101 > 0100: +3 = 1000
	s24 (4)	0000 0001 0000	101010000	corrimiento a la izquierda
26	s22 (2)	0000 0001 0001	101010000	
	s23 (3)	0000 0001 0001	101010000	ningún dígito > 4
	s24 (4)	0000 0010 0010	010100000	corrimiento a la izquierda
27	s22 (2)	0000 0010 0010	010100000	
	s23 (3)	0000 0010 0010	010100000	ningún dígito > 4
	s24 (4)	0000 0100 0100	101000000	corrimiento a la izquierda
28	s22 (2)	0000 0100 0101	101000000	

cont	Estado (paso)	BCD: C D U	bin(31..23)	Comentario
	s23 (3)	0000 0100 1000	101000000	BCD(0) = 0101 > 0100: +3 = 1000
	s24 (4)	0000 1001 0000	010000000	corrimiento a la izquierda
29	s22 (2)	0000 1001 0000	010000000	
	s23 (3)	0000 1100 0000	010000000	BCD(1) = 1001 > 0100: +3 = 1100
	s24 (4)	0001 1000 0000	100000000	corrimiento a la izquierda
30	s22 (2)	0001 1000 0001	100000000	
	s23 (3)	0001 1011 0001	100000000	BCD(1) = 1000 > 0100: +3 = 1011
	s24 (4)	0011 0110 0010	000000000	corrimiento a la izquierda
31	s22 (2)	0011 0110 0010	000000000	último PASO 2 = PASO 5, va a s25

Resultado: BCD = 0011 0110 0010 = 3 6 2.

9. División por restas sucesivas

La división también está dentro del ASM y usa la ALU. Divide magnitudes: residuo = |t3| y divisor = |p|; el signo del resultado (sq = signo(t3) xor signo(p)) se aplica al escribir en la RAM.

- s15: cociente ← 0; residuo ← |t3| (la ALU calcula 0 − t3 si t3 es negativo).
- s16: la ALU calcula residuo − |p|. Si el resultado no es negativo (bit 49 = 0), residuo ← resultado y cociente ← cociente + 1, y el ASM se queda en s16. Si es negativo, pasa a s17.
- s17: RAM(i,j) ← cociente, o 0 − cociente (en la ALU) si sq = 1.

Tarda |cociente| + 1 ciclos en s16. Como la división de Bareiss es exacta, al salir el residuo es 0.

10. Señales del ASM

10.1 Señales de control por estado (ASM → bloques)

Son asignaciones concurrentes de src/asm_bareiss.vhd que dependen del estado. Los estados que no aparecen no usan estos bloques.

Estado	RAM (lectura / escritura)	ALU (M s1 s0 = op)	Otros bloques
s2	escribe (i,j) ← a	—	ROM: dira
s3	—	—	banco: clr_banco = 1
s4	lee (k,k)	—	—
s5	lee (i,k)	—	—
s6	lee (k,j)	—	—
s7	lee (i,j); escribe (i,j) ← tmp1	—	—
s8	escribe (k,j) ← tmp2	—	—
s9	—	—	banco: we_banco, dir_banco = k, dato = piv
s10	lee (i,j)	—	multiplicador: carga mul_a, mul_b
s11	lee (i,k)	—	multiplicador: t1 ← producto
s12	lee (k,j)	—	—
s13	—	—	multiplicador: t2 ← producto
s14	—	110: t1 − t2	—
s15	—	110: 0 − t3	—
s16	—	110: residuo − |p|	—
s17	escribe (i,j) ← ±cociente	110: 0 − cociente	—
s18	—	110: 0 − piv	—
s19	lee (n,n)	110: 0 − RAM(n,n)	banco: we_banco, dir_banco = n, dato = ±RAM(n,n)
s20	—	110: 0 − val	—
s23	—	101: dígito + 3	—
s26	—	—	final = 1

10.2 Señales de estado (bloques → ASM)

Señal	Origen	Se pregunta en
inicio	Pulso del antirrebote	s0, s26
RAM(k,k) = 0, RAM(i,k) = 0	Comparador de q con 0	s4, s5
i = n, j = n	Comparadores de 3 bits	s2, s5, s8, s17
k + 1 = n	Contador k y comparador	s18
signo(t3) = t3(49)	Registro t3	s15
resultado(49)	Signo de la salida de la ALU (residuo − |p| < 0)	s16
sq	Registro del signo del cociente	s17
signo(piv) = piv(31)	Registro piv	s18
signo	Registro de paridad de intercambios	s19
signo(val) = val(31)	Multiplexor val	s20

Señal	Origen	Se pregunta en
cont = 31	Contador cont	s22
dígito(d) > 4	Comparador de 4 bits a la salida del multiplexor de dígitos	s23
d = 9	Contador d	s23
conv_piv	Registro: 0 = convierte det, 1 = convierte pivote	s20, s25
dir_p ≠ dir_ant	Comparador de 3 bits	s26

10.3 Registros del ASM y anchos

Registro	Bits	Uso
st	5 (27 estados)	Estado del ASM
dira_r	5	Contador de dirección de la ROM
n, i, j, k	3 c/u	Tamaño de la matriz y contadores de fila, columna y paso
piv, p, pabs	32 c/u	Pivote actual, pivote anterior y |p| (divisor)
tmp1, tmp2	32 c/u	Intercambio de filas
mul_a, mul_b	25 c/u	Entradas del multiplicador
t1, t2, t3	50 c/u	Productos y numerador de la fórmula (1)
residuo, cociente	50, 32	División por restas sucesivas (cociente es un contador)
sq, signo, conv_piv, neg	1 c/u	Signo del cociente, paridad de intercambios, qué se convierte, signo convertido
det_bin	32	Determinante en binario
bin, BCD	32, 40	Registros binario y decimal del XS3
cont, d	5, 4	Contadores del XS3: bits procesados y dígito revisado
dir_ant	3	dir_p ya convertido
det, pivote	40 c/u	Salidas en BCD

Los contadores (dira, i, j, k, cociente, cont, d) son registros que se incrementan; las operaciones sobre los datos de la matriz y del XS3 pasan por la ALU.

Por la identidad de Sylvester, cada valor de la RAM es el determinante de una submatriz de A, y la desigualdad de Hadamard lo acota: $|\det| \le (32\cdot\sqrt{m})^m$ para una submatriz m×m.

- Operandos del multiplicador: submatrices de hasta 4×4, $|x| < 2^{24}$. Caben en 25 bits con signo.
- Determinante 5×5: $|\det| \le 1{,}88 \times 10^9 < 2^{31}$. RAM, det_bin, cociente y bin son de 32 bits.
- Numerador: $|t3| \le 3{,}2 \times 10^{14} < 2^{49}$. t1, t2, t3, residuo y la ALU son de 50 bits.
- BCD: 32 bits binarios necesitan 10 dígitos. El display muestra 7 y LD1 avisa si hay más.

11. Interfaz con la tarjeta

Divisor de frecuencia

El oscilador de la Nexys A7 es de 100 MHz (pin E3, sección 5 del manual de la tarjeta). El bloque `divisor_frecuencia` genera un pulso de un ciclo cada 1 ms (cuenta 100 000 ciclos) y una onda cuadrada de periodo 1 s (cambia cada 500 ms) que se ve en LD15. Los pulsos se usan como habilitación: todo el diseño trabaja con el mismo reloj, sin relojes derivados.

El procesador trabaja a 100 MHz y no usa el divisor. Con un reloj de 1 s, la matriz 3×3 de prueba (822 ciclos) tardaría unos 14 minutos y la 5×5 (4 583 ciclos) más de una hora.

Antirrebote (uno solo)

El diseño tiene un solo antirrebote, para el botón de inicio. El bloque `antirrebote` sincroniza el botón con dos flip-flops, lo muestrea cada 1 ms en un registro de 8 bits y cambia su nivel estable solo con 8 muestras iguales (8 ms sin rebote). Entrega un pulso de un ciclo en el flanco de subida del nivel estable, así una pulsación arranca un solo cálculo.

Sincronizador y display

Los switches dir_p no llevan antirrebote: el bloque `sincronizador` solo los pasa por dos flip-flops para evitar metaestabilidad. El bloque `display_8dig` cambia de dígito con cada pulso de 1 ms (barrido de 8 ms, 125 Hz): AN7 muestra el signo y AN6..AN0 los 7 dígitos. En el tope, un multiplexor elige det si dir_p = 0 y pivote si dir_p = 1 a 5, y LD1 se enciende si los dígitos 7 a 9 del valor mostrado no son cero.

12. Tiempo de ejecución

- Carga de la matriz (s0 a s3): $n^2 + 3$ ciclos.
- Por cada elemento actualizado: s10 a s15 y s17 (7 ciclos) más |cociente| + 1 ciclos de división en s16.
- XS3: unos 375 ciclos por conversión (s20 a s25). Al terminar el cálculo se hacen dos (det y pivote).

La división por restas sucesivas domina el tiempo cuando los valores son grandes. Las matrices de prueba terminan en menos de 50 µs, salvo la matriz 7 (valores cercanos a ±32), que tarda 19 millones de ciclos (0,19 s). Con los valores de la clase («del orden de cientos de miles») el tiempo es de microsegundos.

13. Implementación en ISE 14.7 y la Nexys A7

13.1 Proyecto

- File → New Project. Top-level source type: HDL. Family: Artix7 · Device: XC7A100T · Package: CSG324 · Speed: −1 · Synthesis: XST · Simulator: ISim · Preferred language: VHDL. ISE solo soporta la Nexys A7-100T.
- Add Source: todos los .vhd de src/ y src/nexys_a7.ucf (Implementation); sim/pkg_prueba.vhd y los testbench de sim/ (Simulation). Si ISE no elige det_bareiss como tope, usen «Set as Top Module».
- Synthesize (View RTL Schematic muestra los bloques) → Implement Design → Generate Programming File. Revisen que se cumpla la restricción de 10 ns (TS_sys_clk).
- Programación: la Nexys A7 no funciona con Digilent Adept. Usen iMPACT de ISE o el Hardware Manager de Vivado (Lab Edition) con el archivo .bit.

13.2 Cambiar la matriz (día de la evaluación)

- Opción 1: editar src/rom_matriz.vhd. La línea de 00h es n; las siguientes son a11, a21, ..., an1, a12, ... en complemento a 2 de 6 bits.
- Opción 2: `python3 sim/modelo_referencia.py rom "2 1 3; 1 4 2; 3 2 5"` escribe el archivo e imprime el determinante y los pivotes esperados.
- Simular con sim/tb_procesador.vhd: lee la ROM, calcula los valores esperados y los compara, sin modificar el testbench.
- Sintetizar y volver a programar la FPGA.

13.3 Operación en la tarjeta

- Pulsar BTNU (rst). Poner SW2..SW0 en 000.
- Pulsar BTNC (inicio). Cuando termina, LD0 (final) se enciende y el display muestra el determinante.
- Poner SW2..SW0 en 1 a n para ver el pivote k; con k = n se ve otra vez el determinante.
- Si LD1 está encendido, el valor tiene más de 7 dígitos y se ven los 7 de abajo. LD15 parpadea cada segundo.

13.4 Matrices de prueba

#	n	Matriz A	det	Pivotes (dir_p = 1..n)	Qué prueba
0	2	7 -3 -3 5	26	7, 26	Caso básico 2×2
1	3	2 1 3 1 4 2 3 2 5	3	2, 7, 3	Ejemplo del documento
2	4	0 2 1 3 2 1 0 1 1 0 3 2 3 1 2 4	-22	2, 4, 13, -22	a11 = 0: intercambio en k = 1
3	5	4 -2 1 3 0 -2 6 2 -1 5 1 2 -7 4 2 3 -1 4 5 -3 0 5 2 -3 8	362	4, 20, -170, -639, 362	5×5 general (ROM por defecto)
4	3	-2 4 1 4 -8 3 1 3 5	50	-2, -10, 50	Pivote negativo; intercambio en k = 2
5	4	1 2 3 0 2 4 1 5 3 1 -2 4 0 5 4 -3	150	1, -5, 25, 150	Pivote cero en k = 2 (p ≠ 1)
6	5	0 0 0 0 0 0 3 1 2 4 0 1 5 -2 1 0 2 -2 6 3 0 4 1 3 7	0	0, 0, 0, 0, 0	Columna 1 en cero: singular
7	5	30 27 24 -29 -31 27 -21 28 29 -24 24 28 2 -28 -28 -29 29 -28 -27 28 -31 -24 -28 28 -30	-17 121 828	30, -1359, 22146, 277564, -17121828	Valores extremos; desborde del display

src/rom_matriz.vhd trae la matriz 3. Para cargar otra de la lista: `python3 sim/modelo_referencia.py rom <número>`.

13.5 Pines (src/nexys_a7.ucf)

Señal	Pines
clk	E3 (100 MHz)
rst / inicio	M18 (BTNU) / N17 (BTNC)
dir_p(0..2)	J15 (SW0), L16 (SW1), M13 (SW2)
final / desborde / latido	H17 (LD0) / K15 (LD1) / V11 (LD15)
seg(0..6) = CA..CG, dp	T10, R10, K16, K13, P15, T11, L18; DP = H15
an(0..7)	J17, J18, T9, J14, P14, T14, K2, U13

14. Simulación

- **sim/tb_alu.vhd** prueba las 8 operaciones de la ALU con valores positivos, negativos y extremos.
- **sim/tb_procesador.vhd** (con sim/pkg_prueba.vhd) lee la matriz de la ROM, calcula el determinante y los pivotes esperados con un modelo de Bareiss en VHDL, pulsa inicio, compara det, recorre dir_p = 1 a n comparando cada pivote, comprueba que el pivote n sea el det y repite todo con un segundo inicio.
- **sim/probar_matrices.py** escribe cada una de las 8 matrices de prueba en una ROM temporal y corre ese testbench con GHDL.
- **sim/tb_det_bareiss.vhd** prueba la tarjeta completa con tiempos reales: pulsa BTNC con rebotes, comprueba que hay un solo cálculo y lee los 8 dígitos decodificando an y seg con dir_p = 0, 2 y n.

#	n	det esperado	det del procesador	Ciclos hasta final	Resultado
0	2	' 0000000026'	' 0000000026'	795	OK
1	3	' 0000000003'	' 0000000003'	822	OK
2	4	'-0000000022'	'-0000000022'	1 003	OK
3	5	' 0000000362'	' 0000000362'	4 583	OK
4	3	' 0000000050'	' 0000000050'	898	OK
5	4	' 0000000150'	' 0000000150'	1 184	OK
6	5	' 0000000000'	' 0000000000'	783	OK
7	5	'-0017121828'	'-0017121828'	19 013 805	OK

Todos los pivotes también coinciden con el modelo, y en la prueba de la tarjeta el display mostró « 0000362» con un solo cálculo después de la pulsación con rebotes (ver sim/resultado_ghdl.txt). Para comprobar que las pruebas detectan errores se probaron tres fallas en una copia del diseño: el XS3 sumando 2 en lugar de 3, el último pivote sin el signo de los intercambios (matriz 2) y la resta de la ALU sin el acarreo de entrada. Las tres se detectaron.

15. Limitaciones y verificación pendiente

- **No se ejecutó ISE.** El diseño se verificó con GHDL 4.1 (simulación con `-fsynopsys -fexplicit` y síntesis genérica). Hay que sintetizar en ISE 14.7 y revisar el reporte de tiempos a 100 MHz. El camino más largo pasa por la ALU de 50 bits; si no se cumplen los 10 ns, generen un reloj de 50 MHz con el Clocking Wizard y pongan PERIOD = 20 ns.

- **Contadores.** Los índices (dira, i, j, k), el cociente y los contadores del XS3 (cont, d) son contadores propios; no pasan por la ALU. Si el profesor quiere que también usen la ALU (A + 1), cada incremento necesita un estado más.
- **Multiplicador.** Es una unidad aparte, combinacional, que usa el paquete `std_logic_signed` sobre std_logic_vector (no el tipo signed). Si el profesor quiere un multiplicador por sumas y desplazamientos que use la ALU, hay que cambiarlo.
- **Salidas en BCD.** det y pivote salen del procesador ya convertidos por el XS3. Si el profesor las quiere en binario, los valores existen dentro del ASM (det_bin y el banco).
- **RAM.** Es el modelo del curso con 64 palabras en lugar de 32 y contenido inicial en cero.
- **Display de 7 dígitos.** Valores hasta ±9 999 999; para valores mayores se enciende LD1.
- **Valor de n.** El diseño supone $2 \leq n \leq 5$ en la ROM y no lo valida.

Anexo A. Código VHDL

src/det_bareiss.vhd

```
 1  ----------------------------------------------------------------------------------
 2  -- Universidad Distrital Francisco Jose de Caldas
 3  -- Analisis y Diseno de Microprocesadores - Taller 5
 4  --
 5  -- TOPE PARA LA NEXYS A7-100T (descripcion estructural)
 6  --
 7  --   divisor_frecuencia -> tick de 1 ms y onda de 1 s
 8  --   antirrebote        -> un pulso de inicio por cada pulsacion de BTNC
 9  --   sincronizador      -> switches dir_p (SW2..SW0)
10  --   procesador         -> ASM de Bareiss con sus bloques (ROM, RAM, division, XS3...)
11  --   display_8dig       -> muestra det (dir_p = 0) o el pivote dir_p (1..5)
12  --
13  -- Herramienta: Xilinx ISE 14.7. Reloj de la tarjeta: 100 MHz (pin E3).
14  ----------------------------------------------------------------------------------
15  library IEEE;
16  use IEEE.STD_LOGIC_1164.ALL;
17
18  entity det_bareiss is
19      Port ( clk      : in  STD_LOGIC;                      -- 100 MHz
20             rst      : in  STD_LOGIC;                      -- BTNU
21             inicio   : in  STD_LOGIC;                      -- BTNC
22             dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);  -- SW2..SW0
23             final    : out STD_LOGIC;                      -- LD0
24             desborde : out STD_LOGIC;                      -- LD1: mas de 7 digitos
25             latido   : out STD_LOGIC;                      -- LD15: onda de 1 s
26             an       : out STD_LOGIC_VECTOR (7 downto 0);
27             seg      : out STD_LOGIC_VECTOR (6 downto 0);
28             dp       : out STD_LOGIC);
29  end det_bareiss;
30
31  architecture Estructural of det_bareiss is
32
33  component divisor_frecuencia
34      Port ( clk, rst : in  STD_LOGIC;
35             tick_1ms : out STD_LOGIC;
36             onda_1s  : out STD_LOGIC);
37  end component;
38
39  component antirrebote
40      Port ( clk, rst : in  STD_LOGIC;
41             tick     : in  STD_LOGIC;
42             boton    : in  STD_LOGIC;
43             pulso    : out STD_LOGIC);
44  end component;
45
46  component sincronizador
47      Port ( clk, rst : in  STD_LOGIC;
48             entrada  : in  STD_LOGIC_VECTOR (2 downto 0);
49             salida   : out STD_LOGIC_VECTOR (2 downto 0));
50  end component;
51
52  component procesador
53      Port ( clk, rst : in  STD_LOGIC;
54             inicio   : in  STD_LOGIC;
55             dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);
56             det      : out STD_LOGIC_VECTOR (39 downto 0);
57             det_neg  : out STD_LOGIC;
58             pivote   : out STD_LOGIC_VECTOR (39 downto 0);
59             piv_neg  : out STD_LOGIC;
60             final    : out STD_LOGIC);
61  end component;
62
63  component display_8dig
64      Port ( clk, rst : in  STD_LOGIC;
65             tick     : in  STD_LOGIC;
66             bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
67             neg      : in  STD_LOGIC;
68             an       : out STD_LOGIC_VECTOR (7 downto 0);
69             seg      : out STD_LOGIC_VECTOR (6 downto 0);
70             dp       : out STD_LOGIC);
71  end component;
72
73  constant CERO12 : STD_LOGIC_VECTOR (11 downto 0) := x"000";
74
75  signal tick_1ms, pulso_inicio : STD_LOGIC;
76  signal dir_p_s     : STD_LOGIC_VECTOR (2 downto 0);
77  signal det, pivote, mostrar : STD_LOGIC_VECTOR (39 downto 0);
78  signal det_neg, piv_neg, mostrar_neg : STD_LOGIC;
79
80  begin
81
```

```
U_FREC : divisor_frecuencia port map (clk => clk, rst => rst, tick_1ms => tick_1ms, onda_1s => latido);

U_ANTI : antirrebote port map (clk => clk, rst => rst, tick => tick_1ms, boton => inicio, pulso => pulso_inicio);

U_SINC : sincronizador port map (clk => clk, rst => rst, entrada => dir_p, salida => dir_p_s);

U_PROC : procesador port map (
    clk => clk, rst => rst, inicio => pulso_inicio, dir_p => dir_p_s,
    det => det, det_neg => det_neg, pivote => pivote, piv_neg => piv_neg, final => final);

-- Que se muestra: dir_p = 0 -> determinante ; dir_p = 1..5 -> pivote dir_p
mostrar     <= det     when dir_p_s = "000" else pivote;
mostrar_neg <= det_neg when dir_p_s = "000" else piv_neg;
desborde    <= '0' when mostrar(39 downto 28) = CERO12 else '1';

U_DISP : display_8dig port map (
    clk => clk, rst => rst, tick => tick_1ms, bcd => mostrar(27 downto 0), neg => mostrar_neg,
    an => an, seg => seg, dp => dp);

end Estructural;
```

src/procesador.vhd

```
--------------------------------------------------------------------------------
-- PROCESADOR: unidad de control (ASM) + camino de datos (descripcion estructural)
--
-- Caja negra:
--   entradas : clk, rst, inicio, dir_p (direccion del pivote, 1..n)
--   salidas  : det (BCD) + det_neg, pivote (BCD) + piv_neg, final
--
-- Bloques internos (cada uno en su archivo):
--   asm_bareiss    ASM de Bareiss con la division y el XS3 integrados
--   rom_matriz     ROM con la matriz (direccion dira, dato a)
--   RAM_memory     RAM de trabajo (modelo del curso)
--   multiplicador  unico multiplicador
--   alu            unica ALU (sumador/restador y operaciones logicas)
--   banco_pivotes  banco de registros de los pivotes
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity procesador is
    Port ( clk, rst : in  STD_LOGIC;
           inicio   : in  STD_LOGIC;
           dir_p    : in  STD_LOGIC_VECTOR (2 downto 0);
           det      : out STD_LOGIC_VECTOR (39 downto 0);
           det_neg  : out STD_LOGIC;
           pivote   : out STD_LOGIC_VECTOR (39 downto 0);
           piv_neg  : out STD_LOGIC;
           final    : out STD_LOGIC);
end procesador;

architecture Estructural of procesador is

component asm_bareiss
    Port ( clk, rst   : in  STD_LOGIC;
           inicio     : in  STD_LOGIC;
           dir_p      : in  STD_LOGIC_VECTOR (2 downto 0);
           dira       : out STD_LOGIC_VECTOR (4 downto 0);
           a          : in  STD_LOGIC_VECTOR (5 downto 0);
           rd_add     : out STD_LOGIC_VECTOR (5 downto 0);
           wr_add     : out STD_LOGIC_VECTOR (5 downto 0);
           we         : out STD_LOGIC;
           data       : out STD_LOGIC_VECTOR (31 downto 0);
           q          : in  STD_LOGIC_VECTOR (31 downto 0);
           mul_a      : out STD_LOGIC_VECTOR (24 downto 0);
           mul_b      : out STD_LOGIC_VECTOR (24 downto 0);
           producto   : in  STD_LOGIC_VECTOR (49 downto 0);
           alu_a      : out STD_LOGIC_VECTOR (49 downto 0);
           alu_b      : out STD_LOGIC_VECTOR (49 downto 0);
           alu_m      : out STD_LOGIC;
           alu_s1     : out STD_LOGIC;
           alu_s0     : out STD_LOGIC;
           resultado  : in  STD_LOGIC_VECTOR (49 downto 0);
           clr_banco  : out STD_LOGIC;
           we_banco   : out STD_LOGIC;
           dir_banco  : out STD_LOGIC_VECTOR (2 downto 0);
           dato_banco : out STD_LOGIC_VECTOR (31 downto 0);
           piv_banco  : in  STD_LOGIC_VECTOR (31 downto 0);
           det        : out STD_LOGIC_VECTOR (39 downto 0);
           det_neg    : out STD_LOGIC;
           pivote     : out STD_LOGIC_VECTOR (39 downto 0);
           piv_neg    : out STD_LOGIC;
           final      : out STD_LOGIC);
end component;

component rom_matriz
```

```
    Port ( dira : in  STD_LOGIC_VECTOR (4 downto 0);
           a    : out STD_LOGIC_VECTOR (5 downto 0));
end component;

component RAM_memory
    port ( c      : in  std_logic;
           data   : in  std_logic_vector (31 downto 0);
           wr_add : in  std_logic_vector (5 downto 0);
           rd_add : in  std_logic_vector (5 downto 0);
           we     : in  std_logic;
           q      : out std_logic_vector (31 downto 0));
end component;

component multiplicador
    Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
           b : in  STD_LOGIC_VECTOR (24 downto 0);
           p : out STD_LOGIC_VECTOR (49 downto 0));
end component;

component alu
    Port ( A  : in  STD_LOGIC_VECTOR (49 downto 0);
           B  : in  STD_LOGIC_VECTOR (49 downto 0);
           M  : in  STD_LOGIC;
           s1 : in  STD_LOGIC;
           s0 : in  STD_LOGIC;
           S  : out STD_LOGIC_VECTOR (49 downto 0));
end component;

component banco_pivotes
    Port ( clk, rst : in  STD_LOGIC;
           clr      : in  STD_LOGIC;
           we       : in  STD_LOGIC;
           dir_w    : in  STD_LOGIC_VECTOR (2 downto 0);
           dato     : in  STD_LOGIC_VECTOR (31 downto 0);
           dir_r    : in  STD_LOGIC_VECTOR (2 downto 0);
           q        : out STD_LOGIC_VECTOR (31 downto 0));
end component;

-- ROM
signal dira : STD_LOGIC_VECTOR (4 downto 0);
signal a    : STD_LOGIC_VECTOR (5 downto 0);
-- RAM
signal rd_add, wr_add : STD_LOGIC_VECTOR (5 downto 0);
signal we             : STD_LOGIC;
signal data, q        : STD_LOGIC_VECTOR (31 downto 0);
-- multiplicador
signal mul_a, mul_b : STD_LOGIC_VECTOR (24 downto 0);
signal producto     : STD_LOGIC_VECTOR (49 downto 0);
-- ALU
signal alu_a, alu_b, resultado : STD_LOGIC_VECTOR (49 downto 0);
signal alu_m, alu_s1, alu_s0   : STD_LOGIC;
-- banco de pivotes
signal clr_banco, we_banco   : STD_LOGIC;
signal dir_banco             : STD_LOGIC_VECTOR (2 downto 0);
signal dato_banco, piv_banco : STD_LOGIC_VECTOR (31 downto 0);

begin

U_ASM : asm_bareiss port map (
    clk => clk, rst => rst, inicio => inicio, dir_p => dir_p,
    dira => dira, a => a,
    rd_add => rd_add, wr_add => wr_add, we => we, data => data, q => q,
    mul_a => mul_a, mul_b => mul_b, producto => producto,
    alu_a => alu_a, alu_b => alu_b, alu_m => alu_m, alu_s1 => alu_s1, alu_s0 => alu_s0,
    resultado => resultado,
    clr_banco => clr_banco, we_banco => we_banco, dir_banco => dir_banco,
    dato_banco => dato_banco, piv_banco => piv_banco,
    det => det, det_neg => det_neg, pivote => pivote, piv_neg => piv_neg,
    final => final);

U_ROM : rom_matriz port map (dira => dira, a => a);

U_RAM : RAM_memory port map (
    c => clk, data => data, wr_add => wr_add, rd_add => rd_add, we => we, q => q);

U_MUL : multiplicador port map (a => mul_a, b => mul_b, p => producto);

U_ALU : alu port map (
    A => alu_a, B => alu_b, M => alu_m, s1 => alu_s1, s0 => alu_s0, S => resultado);

U_BANCO : banco_pivotes port map (
    clk => clk, rst => rst, clr => clr_banco, we => we_banco, dir_w => dir_banco,
    dato => dato_banco, dir_r => dir_p, q => piv_banco);

end Estructural;
```

src/asm_bareiss.vhd

```
----------------------------------------------------------------------------------
-- ASM DEL PROCESADOR: algoritmo de Bareiss con la division por restas
-- sucesivas y el algoritmo XS3 integrados (estados s0 a s26)
--
-- Descripcion de alto nivel del diagrama ASM: un "when" por estado, las cajas
-- de decision son "if" y las salidas condicionales son las asignaciones dentro
-- del "if". Los bloques del camino de datos (ROM, RAM, multiplicador, ALU y
-- banco de pivotes) estan fuera de este archivo: el ASM les entrega
-- direcciones, datos, operandos y la operacion, y lee sus resultados.
--
-- Todas las operaciones sobre datos (resta t1 - t2, division, valores
-- absolutos y el +3 del XS3) pasan por la unica ALU.
-- Solo std_logic y std_logic_vector. Los estados son constantes.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity asm_bareiss is
    Port ( clk, rst   : in  STD_LOGIC;
           inicio     : in  STD_LOGIC;
           dir_p      : in  STD_LOGIC_VECTOR (2 downto 0);
           -- ROM
           dira       : out STD_LOGIC_VECTOR (4 downto 0);
           a          : in  STD_LOGIC_VECTOR (5 downto 0);
           -- RAM
           rd_add     : out STD_LOGIC_VECTOR (5 downto 0);
           wr_add     : out STD_LOGIC_VECTOR (5 downto 0);
           we         : out STD_LOGIC;
           data       : out STD_LOGIC_VECTOR (31 downto 0);
           q          : in  STD_LOGIC_VECTOR (31 downto 0);
           -- multiplicador
           mul_a      : out STD_LOGIC_VECTOR (24 downto 0);
           mul_b      : out STD_LOGIC_VECTOR (24 downto 0);
           producto   : in  STD_LOGIC_VECTOR (49 downto 0);
           -- ALU
           alu_a      : out STD_LOGIC_VECTOR (49 downto 0);
           alu_b      : out STD_LOGIC_VECTOR (49 downto 0);
           alu_m      : out STD_LOGIC;
           alu_s1     : out STD_LOGIC;
           alu_s0     : out STD_LOGIC;
           resultado  : in  STD_LOGIC_VECTOR (49 downto 0);
           -- banco de pivotes
           clr_banco  : out STD_LOGIC;
           we_banco   : out STD_LOGIC;
           dir_banco  : out STD_LOGIC_VECTOR (2 downto 0);
           dato_banco : out STD_LOGIC_VECTOR (31 downto 0);
           piv_banco  : in  STD_LOGIC_VECTOR (31 downto 0);   -- banco(dir_p)
           -- salidas del procesador
           det        : out STD_LOGIC_VECTOR (39 downto 0);   -- |det| en BCD
           det_neg    : out STD_LOGIC;
           pivote     : out STD_LOGIC_VECTOR (39 downto 0);   -- |pivote(dir_p)| en BCD
           piv_neg    : out STD_LOGIC;
           final      : out STD_LOGIC);
end asm_bareiss;

architecture Behavioral of asm_bareiss is

----------------------------------------------------------------------------------
-- Estados
----------------------------------------------------------------------------------
-- s0        espera de inicio
-- s1..s3    lectura de n, copia ROM -> RAM, inicializacion
-- s4..s9    pivote: lectura, busqueda de fila, intercambio, banco(k)
-- s10..s14  productos y resta t1 - t2
-- s15..s16  division por restas sucesivas |t3| / |p|
-- s17       escritura de a(i,j) y avance de i, j
-- s18       p = pivote actual, k = k + 1
-- s19       det = +/- a(n,n) ; banco(n) = det
-- s20       valor a convertir: det (1a pasada) o pivote(dir_p) (2a pasada)
-- s21..s24  algoritmo XS3 (pasos 1 a 5)
-- s25       guarda el BCD en det o en pivote
-- s26       final = 1
constant s0  : STD_LOGIC_VECTOR (4 downto 0) := "00000";
constant s1  : STD_LOGIC_VECTOR (4 downto 0) := "00001";
constant s2  : STD_LOGIC_VECTOR (4 downto 0) := "00010";
constant s3  : STD_LOGIC_VECTOR (4 downto 0) := "00011";
constant s4  : STD_LOGIC_VECTOR (4 downto 0) := "00100";
constant s5  : STD_LOGIC_VECTOR (4 downto 0) := "00101";
constant s6  : STD_LOGIC_VECTOR (4 downto 0) := "00110";
constant s7  : STD_LOGIC_VECTOR (4 downto 0) := "00111";
constant s8  : STD_LOGIC_VECTOR (4 downto 0) := "01000";
constant s9  : STD_LOGIC_VECTOR (4 downto 0) := "01001";
constant s10 : STD_LOGIC_VECTOR (4 downto 0) := "01010";
constant s11 : STD_LOGIC_VECTOR (4 downto 0) := "01011";
constant s12 : STD_LOGIC_VECTOR (4 downto 0) := "01100";
constant s13 : STD_LOGIC_VECTOR (4 downto 0) := "01101";
constant s14 : STD_LOGIC_VECTOR (4 downto 0) := "01110";
```

```
 89  constant s15 : STD_LOGIC_VECTOR (4 downto 0) := "01111";
 90  constant s16 : STD_LOGIC_VECTOR (4 downto 0) := "10000";
 91  constant s17 : STD_LOGIC_VECTOR (4 downto 0) := "10001";
 92  constant s18 : STD_LOGIC_VECTOR (4 downto 0) := "10010";
 93  constant s19 : STD_LOGIC_VECTOR (4 downto 0) := "10011";
 94  constant s20 : STD_LOGIC_VECTOR (4 downto 0) := "10100";
 95  constant s21 : STD_LOGIC_VECTOR (4 downto 0) := "10101";
 96  constant s22 : STD_LOGIC_VECTOR (4 downto 0) := "10110";
 97  constant s23 : STD_LOGIC_VECTOR (4 downto 0) := "10111";
 98  constant s24 : STD_LOGIC_VECTOR (4 downto 0) := "11000";
 99  constant s25 : STD_LOGIC_VECTOR (4 downto 0) := "11001";
100  constant s26 : STD_LOGIC_VECTOR (4 downto 0) := "11010";
101
102  signal st : STD_LOGIC_VECTOR (4 downto 0);
103
104  ---------------------------------------------------------------------------
105  -- Operaciones de la ALU (M s1 s0)
106  ---------------------------------------------------------------------------
107  constant ALU_SUMA  : STD_LOGIC_VECTOR (2 downto 0) := "101";   -- A + B
108  constant ALU_RESTA : STD_LOGIC_VECTOR (2 downto 0) := "110";   -- A - B
109
110  ---------------------------------------------------------------------------
111  -- Constantes
112  ---------------------------------------------------------------------------
113  constant UNO3   : STD_LOGIC_VECTOR (2 downto 0)  := "001";
114  constant CERO18 : STD_LOGIC_VECTOR (17 downto 0) := "000000000000000000";
115  constant CERO32 : STD_LOGIC_VECTOR (31 downto 0) := x"00000000";
116  constant UNO32  : STD_LOGIC_VECTOR (31 downto 0) := x"00000001";
117  constant CERO40 : STD_LOGIC_VECTOR (39 downto 0) := x"0000000000";
118  constant CERO46 : STD_LOGIC_VECTOR (45 downto 0) := (others => '0');
119  constant CERO50 : STD_LOGIC_VECTOR (49 downto 0) := (others => '0');
120  constant TRES50 : STD_LOGIC_VECTOR (49 downto 0) := (1 downto 0 => '1', others => '0');
121
122  ---------------------------------------------------------------------------
123  -- Registros del ASM
124  ---------------------------------------------------------------------------
125  signal dira_r     : STD_LOGIC_VECTOR (4 downto 0);   -- direccion de la ROM
126  signal n, i, j, k : STD_LOGIC_VECTOR (2 downto 0);   -- tamano y contadores
127  signal piv        : STD_LOGIC_VECTOR (31 downto 0);  -- pivote actual a(k,k)
128  signal p          : STD_LOGIC_VECTOR (31 downto 0);  -- pivote anterior
129  signal pabs       : STD_LOGIC_VECTOR (31 downto 0);  -- |p| (divisor)
130  signal signo      : STD_LOGIC;                       -- paridad de intercambios
131  signal tmp1, tmp2 : STD_LOGIC_VECTOR (31 downto 0);  -- intercambio de filas
132  signal mul_a_r, mul_b_r : STD_LOGIC_VECTOR (24 downto 0);
133  signal t1, t2, t3 : STD_LOGIC_VECTOR (49 downto 0);  -- productos y numerador
134  signal residuo    : STD_LOGIC_VECTOR (49 downto 0);  -- division: residuo
135  signal cociente   : STD_LOGIC_VECTOR (31 downto 0);  -- division: cociente
136  signal sq         : STD_LOGIC;                       -- signo del cociente
137  signal det_bin    : STD_LOGIC_VECTOR (31 downto 0);  -- determinante en binario
138  signal conv_piv   : STD_LOGIC;                       -- '0' convierte det, '1' el pivote
139  signal dir_ant    : STD_LOGIC_VECTOR (2 downto 0);   -- dir_p ya convertido
140  signal neg        : STD_LOGIC;                       -- signo del valor convertido
141  signal bin        : STD_LOGIC_VECTOR (31 downto 0);  -- XS3: registro binario
142  signal bcd        : STD_LOGIC_VECTOR (39 downto 0);  -- XS3: registro decimal (10 digitos)
143  signal cont       : STD_LOGIC_VECTOR (4 downto 0);   -- XS3: bits procesados (0..31)
144  signal d          : STD_LOGIC_VECTOR (3 downto 0);   -- XS3: digito revisado (0..9)
145  signal det_bcd, piv_bcd     : STD_LOGIC_VECTOR (39 downto 0);
146  signal det_neg_r, piv_neg_r : STD_LOGIC;
147
148  ---------------------------------------------------------------------------
149  -- Senales combinacionales
150  ---------------------------------------------------------------------------
151  signal dira_mas1, cont_mas1 : STD_LOGIC_VECTOR (4 downto 0);    -- contadores
152  signal i_mas1, j_mas1, k_mas1 : STD_LOGIC_VECTOR (2 downto 0);
153  signal cociente_mas1 : STD_LOGIC_VECTOR (31 downto 0);
154  signal d_mas1  : STD_LOGIC_VECTOR (3 downto 0);
155  signal ext_a   : STD_LOGIC_VECTOR (25 downto 0);                -- extensiones de signo
156  signal a32     : STD_LOGIC_VECTOR (31 downto 0);
157  signal sx_piv, sx_q, sx_val : STD_LOGIC_VECTOR (17 downto 0);
158  signal rd_fila, rd_col, wr_fila : STD_LOGIC_VECTOR (2 downto 0);
159  signal op_alu  : STD_LOGIC_VECTOR (2 downto 0);
160  signal val     : STD_LOGIC_VECTOR (31 downto 0);                -- valor que se convierte
161  signal det_nn  : STD_LOGIC_VECTOR (31 downto 0);                -- +/- a(n,n)
162  signal dig     : STD_LOGIC_VECTOR (3 downto 0);                 -- digito d del BCD
163
164  begin
165
166  ---------------------------------------------------------------------------
167  -- Contadores y extensiones de signo
168  ---------------------------------------------------------------------------
169  dira_mas1     <= dira_r + '1';
170  i_mas1        <= i + '1';
171  j_mas1        <= j + '1';
172  k_mas1        <= k + '1';
173  cociente_mas1 <= cociente + '1';
174  cont_mas1     <= cont + '1';
175  d_mas1        <= d + '1';
176
```

```
ext_a  <= (others => a(5));
a32    <= ext_a & a;
sx_piv <= (others => piv(31));
sx_q   <= (others => q(31));
sx_val <= (others => val(31));

-------------------------------------------------------------------------------
-- Senales hacia la ROM y la RAM
-------------------------------------------------------------------------------
dira <= dira_r;

rd_fila <= k when (st = s4 or st = s6 or st = s12) else      -- RAM(k,k), RAM(k,j)
           n when st = s19 else                              -- RAM(n,n)
           i;
rd_col  <= k when (st = s4 or st = s5 or st = s11) else      -- RAM(k,k), RAM(i,k)
           n when st = s19 else
           j;
rd_add  <= rd_fila & rd_col;

wr_fila <= k when st = s8 else i;                            -- RAM(k,j) en s8
wr_add  <= wr_fila & j;

we   <= '1' when (st = s2 or st = s7 or st = s8 or st = s17) else '0';

data <= a32                   when st = s2 else              -- copia de la ROM
        tmp1                  when st = s7 else              -- intercambio
        tmp2                  when st = s8 else              -- intercambio
        resultado(31 downto 0) when sq = '1' else            -- s17: -cociente
        cociente;                                            -- s17: +cociente

-------------------------------------------------------------------------------
-- Multiplicador
-------------------------------------------------------------------------------
mul_a <= mul_a_r;
mul_b <= mul_b_r;

-------------------------------------------------------------------------------
-- ALU: operandos y operacion segun el estado
-------------------------------------------------------------------------------
alu_a <= t1           when st = s14 else                     -- t1 - t2
         residuo      when st = s16 else                     -- residuo - |p|
         CERO46 & dig when st = s23 else                     -- digito + 3 (XS3)
         CERO50;                                             -- 0 - x (negacion)
alu_b <= t2                when st = s14 else
         t3                when st = s15 else                -- 0 - t3
         CERO18 & pabs     when st = s16 else                -- |p|
         CERO18 & cociente when st = s17 else                -- 0 - cociente
         sx_piv & piv      when st = s18 else                -- 0 - piv
         sx_q & q          when st = s19 else                -- 0 - a(n,n)
         sx_val & val      when st = s20 else                -- 0 - valor
         TRES50            when st = s23 else                -- +3 (XS3)
         CERO50;
op_alu <= ALU_SUMA when st = s23 else ALU_RESTA;
alu_m  <= op_alu(2);
alu_s1 <= op_alu(1);
alu_s0 <= op_alu(0);

-------------------------------------------------------------------------------
-- Banco de pivotes
-------------------------------------------------------------------------------
det_nn     <= resultado(31 downto 0) when signo = '1' else q; -- s19
clr_banco  <= '1' when st = s3 else '0';
we_banco   <= '1' when (st = s9 or st = s19) else '0';
dir_banco  <= n when st = s19 else k;
dato_banco <= det_nn when st = s19 else piv;

-------------------------------------------------------------------------------
-- XS3: valor a convertir y digito d del registro decimal
-------------------------------------------------------------------------------
val <= det_bin when conv_piv = '0' else piv_banco;

with d select
    dig <= bcd(3 downto 0)   when "0000",
           bcd(7 downto 4)   when "0001",
           bcd(11 downto 8)  when "0010",
           bcd(15 downto 12) when "0011",
           bcd(19 downto 16) when "0100",
           bcd(23 downto 20) when "0101",
           bcd(27 downto 24) when "0110",
           bcd(31 downto 28) when "0111",
           bcd(35 downto 32) when "1000",
           bcd(39 downto 36) when "1001",
           "0000"            when others;

-------------------------------------------------------------------------------
-- Salidas del procesador
-------------------------------------------------------------------------------
det     <= det_bcd;
```

```
det_neg <= det_neg_r;
pivote  <= piv_bcd;
piv_neg <= piv_neg_r;
final   <= '1' when st = s26 else '0';

-------------------------------------------------------------------------------
-- DIAGRAMA ASM
-------------------------------------------------------------------------------
process (clk, rst)
begin

if rst = '1' then
    st       <= s0;
    dira_r   <= "00000";
    n <= "000";  i <= "000";  j <= "000";  k <= "000";
    piv      <= CERO32;  p <= CERO32;  pabs <= CERO32;  signo <= '0';
    tmp1     <= CERO32;  tmp2 <= CERO32;
    mul_a_r  <= (others => '0');  mul_b_r <= (others => '0');
    t1       <= CERO50;  t2 <= CERO50;  t3 <= CERO50;
    residuo  <= CERO50;  cociente <= CERO32;  sq <= '0';
    det_bin  <= CERO32;
    conv_piv <= '0';     dir_ant <= "000";  neg <= '0';
    bin      <= CERO32;  bcd <= CERO40;  cont <= "00000";  d <= "0000";
    det_bcd  <= CERO40;  det_neg_r <= '0';
    piv_bcd  <= CERO40;  piv_neg_r <= '0';

elsif clk'event and clk = '1' then

    case st is

        ---------------------------------------------------------------
        -- Espera y carga de la matriz
        ---------------------------------------------------------------
        when s0 =>                              -- final = 0 ; dira = 0
            dira_r <= "00000";
            if inicio = '0' then
                st <= s0;
            else
                st <= s1;
            end if;

        when s1 =>                              -- n = ROM(00h)
            n      <= a(2 downto 0);
            dira_r <= "00001";
            i      <= UNO3;
            j      <= UNO3;
            st     <= s2;

        when s2 =>                              -- RAM(i,j) = ROM(dira), por columnas
            dira_r <= dira_mas1;
            if i = n then
                i <= UNO3;
                if j = n then
                    st <= s3;
                else
                    j  <= j_mas1;
                    st <= s2;
                end if;
            else
                i  <= i_mas1;
                st <= s2;
            end if;

        when s3 =>                              -- inicializacion ; banco = 0
            k        <= UNO3;
            p        <= UNO32;                  -- pivote inicial a(0,0) = 1
            pabs     <= UNO32;
            signo    <= '0';
            conv_piv <= '0';
            st       <= s4;

        ---------------------------------------------------------------
        -- Pivote del paso k
        ---------------------------------------------------------------
        when s4 =>                              -- piv = RAM(k,k)
            piv <= q;
            if q = CERO32 then
                i  <= k_mas1;
                st <= s5;
            else
                st <= s9;
            end if;

        when s5 =>                              -- busca fila i > k con RAM(i,k) /= 0
            if q /= CERO32 then
                j     <= k;
                signo <= not signo;             -- el intercambio cambia el signo
                st    <= s6;
```

```
353             elsif i = n then
354                 det_bin <= CERO32;              -- no hay fila: matriz singular
355                 st      <= s20;
356             else
357                 i  <= i_mas1;
358                 st <= s5;
359             end if;
360
361         when s6 =>                              -- tmp1 = RAM(k,j)
362             tmp1 <= q;
363             st   <= s7;
364
365         when s7 =>                              -- tmp2 = RAM(i,j) ; RAM(i,j) = tmp1
366             tmp2 <= q;
367             st   <= s8;
368
369         when s8 =>                              -- RAM(k,j) = tmp2
370             if j = n then
371                 st <= s4;                       -- vuelve a leer el pivote
372             else
373                 j  <= j_mas1;
374                 st <= s6;
375             end if;
376
377         when s9 =>                              -- banco(k) = piv ; i = j = k + 1
378             i  <= k_mas1;
379             j  <= k_mas1;
380             st <= s10;
381
382         ----------------------------------------------------------------
383         -- a(i,j) = ( a(k,k)*a(i,j) - a(i,k)*a(k,j) ) / p
384         ----------------------------------------------------------------
385         when s10 =>                             -- mul_a = a(k,k) ; mul_b = RAM(i,j)
386             mul_a_r <= piv(24 downto 0);
387             mul_b_r <= q(24 downto 0);
388             st      <= s11;
389
390         when s11 =>                             -- t1 = producto ; mul_a = RAM(i,k)
391             t1      <= producto;
392             mul_a_r <= q(24 downto 0);
393             st      <= s12;
394
395         when s12 =>                             -- mul_b = RAM(k,j)
396             mul_b_r <= q(24 downto 0);
397             st      <= s13;
398
399         when s13 =>                             -- t2 = producto
400             t2 <= producto;
401             st <= s14;
402
403         when s14 =>                             -- t3 = t1 - t2  (ALU)
404             t3 <= resultado;
405             st <= s15;
406
407         ----------------------------------------------------------------
408         -- Division por restas sucesivas: cociente = |t3| / |p|
409         ----------------------------------------------------------------
410         when s15 =>                             -- residuo = |t3| ; cociente = 0
411             cociente <= CERO32;
412             sq       <= t3(49) xor p(31);       -- signo del cociente
413             if t3(49) = '1' then
414                 residuo <= resultado;           -- residuo = 0 - t3  (ALU)
415             else
416                 residuo <= t3;
417             end if;
418             st <= s16;
419
420         when s16 =>                             -- ALU: residuo - |p|
421             if resultado(49) = '0' then         -- residuo - |p| >= 0
422                 residuo  <= resultado;
423                 cociente <= cociente_mas1;
424                 st       <= s16;
425             else
426                 st       <= s17;
427             end if;
428
429         when s17 =>                             -- RAM(i,j) = +/- cociente ; avanza j, i
430             if j = n then
431                 if i = n then
432                     st <= s18;
433                 else
434                     i  <= i_mas1;
435                     j  <= k_mas1;
436                     st <= s10;
437                 end if;
438             else
439                 j  <= j_mas1;
440                 st <= s10;
```

```
441             end if;
442
443         when s18 =>                             -- p = piv ; |p| = |piv| ; k = k + 1
444             p <= piv;
445             if piv(31) = '1' then
446                 pabs <= resultado(31 downto 0); -- |p| = 0 - piv  (ALU)
447             else
448                 pabs <= piv;
449             end if;
450             k <= k_mas1;
451             if k_mas1 = n then                  -- k = n - 1 : ultimo paso
452                 st <= s19;
453             else
454                 st <= s4;
455             end if;
456
457         when s19 =>                             -- det = +/- RAM(n,n) ; banco(n) = det
458             det_bin <= det_nn;
459             st      <= s20;
460
461         ---------------------------------------------------------------
462         -- Algoritmo XS3 (primero det, luego el pivote elegido)
463         ---------------------------------------------------------------
464         when s20 =>                             -- bin = |val| ; neg = signo de val
465             dir_ant <= dir_p;
466             neg     <= val(31);
467             if val(31) = '1' then
468                 bin <= resultado(31 downto 0);  -- bin = 0 - val  (ALU)
469             else
470                 bin <= val;
471             end if;
472             st <= s21;
473
474         when s21 =>                             -- XS3 paso 1: registro decimal = 0
475             bcd  <= CERO40;
476             cont <= "00000";
477             st   <= s22;
478
479         when s22 =>                             -- XS3 paso 2: LSB decimal = MSB binario
480             bcd(0) <= bin(31);
481             if cont = "11111" then
482                 st <= s25;                      -- paso 5: ultimo paso 2 hecho
483             else
484                 d  <= "0000";
485                 st <= s23;
486             end if;
487
488         when s23 =>                             -- XS3 paso 3: si digito(d) > 4, sumar 3 (ALU)
489             if dig > "0100" then
490                 case d is
491                     when "0000" => bcd(3 downto 0)   <= resultado(3 downto 0);
492                     when "0001" => bcd(7 downto 4)   <= resultado(3 downto 0);
493                     when "0010" => bcd(11 downto 8)  <= resultado(3 downto 0);
494                     when "0011" => bcd(15 downto 12) <= resultado(3 downto 0);
495                     when "0100" => bcd(19 downto 16) <= resultado(3 downto 0);
496                     when "0101" => bcd(23 downto 20) <= resultado(3 downto 0);
497                     when "0110" => bcd(27 downto 24) <= resultado(3 downto 0);
498                     when "0111" => bcd(31 downto 28) <= resultado(3 downto 0);
499                     when "1000" => bcd(35 downto 32) <= resultado(3 downto 0);
500                     when "1001" => bcd(39 downto 36) <= resultado(3 downto 0);
501                     when others => null;
502                 end case;
503             end if;
504             if d = "1001" then
505                 st <= s24;
506             else
507                 d  <= d_mas1;
508                 st <= s23;
509             end if;
510
511         when s24 =>                             -- XS3 paso 4: corrimiento a la izquierda
512             bcd  <= bcd(38 downto 0) & '0';
513             bin  <= bin(30 downto 0) & '0';
514             cont <= cont_mas1;
515             st   <= s22;
516
517         when s25 =>                             -- guarda el resultado del XS3
518             if conv_piv = '0' then
519                 det_bcd   <= bcd;
520                 det_neg_r <= neg;
521                 conv_piv  <= '1';
522                 st        <= s20;               -- ahora convierte el pivote
523             else
524                 piv_bcd   <= bcd;
525                 piv_neg_r <= neg;
526                 st        <= s26;
527             end if;
528
```

```
        when s26 =>                             -- final = 1
            dira_r <= "00000";
            if inicio = '1' then
                st <= s1;                       -- nuevo calculo
            elsif dir_p /= dir_ant then
                st <= s20;                      -- cambio dir_p: convierte el nuevo pivote
            else
                st <= s26;
            end if;

        when others =>
            st <= s0;

    end case;
end if;
end process;

end Behavioral;
```

src/alu.vhd

```
----------------------------------------------------------------------------------
-- ALU DEL PROCESADOR (50 bits, combinacional)
--
-- Funcionamiento de la ALU (tabla del curso):
--
--     M  s1  s0  | Operacion
--     0   0   0  | not B
--     0   0   1  | A and B
--     0   1   0  | A or B
--     0   1   1  | A xnor B
--     1   0   0  | A + 1
--     1   0   1  | A + B
--     1   1   0  | A - B
--     1   1   1  | A - 1
--
-- Es el unico sumador/restador del procesador: el ASM de Bareiss la usa para
-- t1 - t2, para la division por restas sucesivas, para los valores absolutos
-- (0 - x) y para el +3 del algoritmo XS3.
--
-- Las cuatro operaciones aritmeticas usan un solo sumador  S = A + Bx + cin:
--     A + 1 : Bx = 0      , cin = 1
--     A + B : Bx = B      , cin = 0
--     A - B : Bx = not B  , cin = 1      (complemento a 2)
--     A - 1 : Bx = 1...1  , cin = 0      (sumar -1)
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity alu is
    Port ( A  : in  STD_LOGIC_VECTOR (49 downto 0);
           B  : in  STD_LOGIC_VECTOR (49 downto 0);
           M  : in  STD_LOGIC;
           s1 : in  STD_LOGIC;
           s0 : in  STD_LOGIC;
           S  : out STD_LOGIC_VECTOR (49 downto 0));
end alu;

architecture Behavioral of alu is

constant CERO50 : STD_LOGIC_VECTOR (49 downto 0) := (others => '0');
constant UNOS50 : STD_LOGIC_VECTOR (49 downto 0) := (others => '1');

signal sel     : STD_LOGIC_VECTOR (2 downto 0);
signal bx      : STD_LOGIC_VECTOR (49 downto 0);   -- segundo operando del sumador
signal cin     : STD_LOGIC;                        -- acarreo de entrada
signal suma    : STD_LOGIC_VECTOR (49 downto 0);   -- unidad aritmetica
signal logica  : STD_LOGIC_VECTOR (49 downto 0);   -- unidad logica

begin

sel <= M & s1 & s0;

-- Unidad aritmetica: un solo sumador
with sel select
    bx <= CERO50 when "100",          -- A + 1
          B      when "101",          -- A + B
          not B  when "110",          -- A - B
          UNOS50 when others;         -- A - 1

cin  <= '1' when (sel = "100" or sel = "110") else '0';

suma <= A + bx + cin;

-- Unidad logica
with sel select
```

```
    logica <= not B      when "000",
              A and B    when "001",
              A or B     when "010",
              A xnor B   when others;

-- Salida: M = 0 logica, M = 1 aritmetica
S <= logica when M = '0' else suma;

end Behavioral;
```

src/multiplicador.vhd

```
----------------------------------------------------------------------------------
-- MULTIPLICADOR CON SIGNO 25 x 25 -> 50 bits (combinacional)
--
-- Unico multiplicador del procesador. Los puertos son std_logic_vector; el
-- paquete std_logic_signed hace que "*" los interprete en complemento a 2.
-- 25 bits alcanzan: en el paso k los operandos son determinantes de
-- submatrices de k x k (k <= 4), menores que 2^24 en valor absoluto.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_signed.all;

entity multiplicador is
    Port ( a : in  STD_LOGIC_VECTOR (24 downto 0);
           b : in  STD_LOGIC_VECTOR (24 downto 0);
           p : out STD_LOGIC_VECTOR (49 downto 0));
end multiplicador;

architecture Behavioral of multiplicador is
begin

p <= a * b;

end Behavioral;
```

src/banco_pivotes.vhd

```
----------------------------------------------------------------------------------
-- BANCO DE REGISTROS DE PIVOTES (5 registros de 32 bits)
--
-- Registro k = pivote a(k,k) usado en el paso k (k = 1..n-1).
-- Registro n = determinante (el ultimo pivote es el determinante).
-- Escritura sincrona en la direccion dir_w; lectura combinacional en dir_r,
-- que viene de los switches (dir_p). Las direcciones 0, 6 y 7 leen 0.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity banco_pivotes is
    Port ( clk, rst : in  STD_LOGIC;
           clr      : in  STD_LOGIC;                      -- borra los 5 registros
           we       : in  STD_LOGIC;
           dir_w    : in  STD_LOGIC_VECTOR (2 downto 0);
           dato     : in  STD_LOGIC_VECTOR (31 downto 0);
           dir_r    : in  STD_LOGIC_VECTOR (2 downto 0);
           q        : out STD_LOGIC_VECTOR (31 downto 0));
end banco_pivotes;

architecture Behavioral of banco_pivotes is

constant CERO32 : STD_LOGIC_VECTOR (31 downto 0) := x"00000000";

signal p1, p2, p3, p4, p5 : STD_LOGIC_VECTOR (31 downto 0);

begin

process (clk, rst)
begin
if rst = '1' then
    p1 <= CERO32;  p2 <= CERO32;  p3 <= CERO32;  p4 <= CERO32;  p5 <= CERO32;
elsif clk'event and clk = '1' then
    if clr = '1' then
        p1 <= CERO32;  p2 <= CERO32;  p3 <= CERO32;  p4 <= CERO32;  p5 <= CERO32;
    elsif we = '1' then
        case dir_w is
            when "001"  => p1 <= dato;
            when "010"  => p2 <= dato;
            when "011"  => p3 <= dato;
            when "100"  => p4 <= dato;
            when "101"  => p5 <= dato;
            when others => null;
        end case;
    end if;
end if;
```

```
end process;

with dir_r select
    q <= p1     when "001",
         p2     when "010",
         p3     when "011",
         p4     when "100",
         p5     when "101",
         CERO32 when others;

end Behavioral;
```

src/ram.vhd

```
----------------------------------------------------------------------------------
-- RAM DE TRABAJO (modelo de RAM del curso)
--
-- Es el modelo del aula con dos cambios de tamano:
--   * 64 palabras en lugar de 32, para que la direccion sea fila(3) & columna(3)
--     y no haya que multiplicar para ubicar a(i,j).
--   * Contenido inicial en cero (solo evita valores 'U' en la simulacion).
-- Escritura sincrona; lectura asincrona por una direccion aparte (rd_add).
-- Es la unica parte del diseno que usa un tipo distinto de std_logic (el
-- arreglo de la memoria) y conv_integer para la linea de direccion.
----------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity RAM_memory is
   port
   (
      c      : in  std_logic;
      data   : in  std_logic_vector (31 downto 0);
      wr_add : in  std_logic_vector (5 downto 0);
      rd_add : in  std_logic_vector (5 downto 0);
      we     : in  std_logic;
      q      : out std_logic_vector (31 downto 0)
   );
end RAM_memory;

architecture rtl of RAM_memory is
   type mem is array(0 to 63) of std_logic_vector(31 downto 0);
   signal ram_blk : mem := (others => (others => '0'));
begin

   process (c, data, wr_add, rd_add, ram_blk)
   begin
      if (c'event and c = '1') then
         if (we = '1') then
            ram_blk(conv_integer(wr_add)) <= data;
         end if;
      end if;
   end process;

   q <= ram_blk(conv_integer(rd_add));

end rtl;
```

src/rom_matriz.vhd

```
----------------------------------------------------------------------------------
-- ROM DE LA MATRIZ (32 x 6 bits, lectura combinacional)
--
-- Hace parte de la arquitectura del procesador: para calcular el determinante
-- de otra matriz se cambia este archivo y se vuelve a programar la FPGA.
-- Archivo generado con: python3 sim/modelo_referencia.py rom ...
--
-- Posicion 00h = n; luego los elementos a(i,j) por columnas (a11, a21, ...),
-- como en la Figura 1 del taller. Datos en complemento a 2 (-32 a 31).
--
-- Matriz (5x5):
--       4   -2    1    3    0
--      -2    6    2   -1    5
--       1    2   -7    4    2
--       3   -1    4    5   -3
--       0    5    2   -3    8
-- Resultado esperado: det = 362
-- Pivotes (dir_p = 1..5): 4, 20, -170, -639, 362
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity rom_matriz is
    Port ( dira : in  STD_LOGIC_VECTOR (4 downto 0);
           a    : out STD_LOGIC_VECTOR (5 downto 0));
```

```
end rom_matriz;

architecture Behavioral of rom_matriz is
begin

with dira select
    a <= "000101" when "00000",   -- 00h: n = 5
         "000100" when "00001",   -- 01h: a11 = 4
         "111110" when "00010",   -- 02h: a21 = -2
         "000001" when "00011",   -- 03h: a31 = 1
         "000011" when "00100",   -- 04h: a41 = 3
         "000000" when "00101",   -- 05h: a51 = 0
         "111110" when "00110",   -- 06h: a12 = -2
         "000110" when "00111",   -- 07h: a22 = 6
         "000010" when "01000",   -- 08h: a32 = 2
         "111111" when "01001",   -- 09h: a42 = -1
         "000101" when "01010",   -- 0Ah: a52 = 5
         "000001" when "01011",   -- 0Bh: a13 = 1
         "000010" when "01100",   -- 0Ch: a23 = 2
         "111001" when "01101",   -- 0Dh: a33 = -7
         "000100" when "01110",   -- 0Eh: a43 = 4
         "000010" when "01111",   -- 0Fh: a53 = 2
         "000011" when "10000",   -- 10h: a14 = 3
         "111111" when "10001",   -- 11h: a24 = -1
         "000100" when "10010",   -- 12h: a34 = 4
         "000101" when "10011",   -- 13h: a44 = 5
         "111101" when "10100",   -- 14h: a54 = -3
         "000000" when "10101",   -- 15h: a15 = 0
         "000101" when "10110",   -- 16h: a25 = 5
         "000010" when "10111",   -- 17h: a35 = 2
         "111101" when "11000",   -- 18h: a45 = -3
         "001000" when "11001",   -- 19h: a55 = 8
         "000000" when others;

end Behavioral;
```

src/divisor_frecuencia.vhd

```
--------------------------------------------------------------------------------
-- DIVISOR DE FRECUENCIA (reloj de la Nexys A7 = 100 MHz, pin E3)
--
--   tick_1ms : pulso de un ciclo cada 1 ms (100 000 ciclos). Lo usan el
--              antirrebote (muestreo del boton) y el display (cambio de digito).
--   onda_1s  : onda cuadrada de periodo 1 s (cambia cada 500 ms).
--
-- Los pulsos se usan como habilitacion (clock enable): todo el diseno trabaja
-- con el mismo reloj de 100 MHz. El procesador no usa este divisor: con un
-- reloj de 1 s una matriz de 3x3 tardaria unos 13 minutos.
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity divisor_frecuencia is
    Port ( clk, rst : in  STD_LOGIC;
           tick_1ms : out STD_LOGIC;
           onda_1s  : out STD_LOGIC);
end divisor_frecuencia;

architecture Behavioral of divisor_frecuencia is

constant FIN_1MS   : STD_LOGIC_VECTOR (16 downto 0) := "11000011010011111";   -- 99 999
constant FIN_500MS : STD_LOGIC_VECTOR (8 downto 0)  := "111110011";           -- 499

signal cuenta_ciclos : STD_LOGIC_VECTOR (16 downto 0);
signal cuenta_ms     : STD_LOGIC_VECTOR (8 downto 0);
signal tick          : STD_LOGIC;
signal onda          : STD_LOGIC;

begin

tick <= '1' when cuenta_ciclos = FIN_1MS else '0';

process (clk, rst)
begin
if rst = '1' then
    cuenta_ciclos <= (others => '0');
    cuenta_ms     <= (others => '0');
    onda          <= '0';
elsif clk'event and clk = '1' then
    if tick = '1' then
        cuenta_ciclos <= (others => '0');
        if cuenta_ms = FIN_500MS then
            cuenta_ms <= (others => '0');
            onda      <= not onda;
        else
            cuenta_ms <= cuenta_ms + '1';
```

```
        end if;
    else
        cuenta_ciclos <= cuenta_ciclos + '1';
    end if;
end if;
end process;

tick_1ms <= tick;
onda_1s  <= onda;

end Behavioral;
```

src/antirrebote.vhd

```
----------------------------------------------------------------------------------
-- ANTIRREBOTE PARA EL BOTON DE INICIO
--
-- 1. Sincroniza el boton con dos flip-flops.
-- 2. Lo muestrea cada 1 ms (tick) en un registro de 8 muestras.
-- 3. El nivel estable cambia a '1' con 8 muestras en '1' (8 ms sin rebote) y
--    a '0' con 8 muestras en '0'.
-- 4. pulso = '1' durante un solo ciclo de reloj cuando el nivel estable pasa
--    de '0' a '1': una pulsacion genera un unico inicio.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity antirrebote is
    Port ( clk, rst : in  STD_LOGIC;
           tick     : in  STD_LOGIC;      -- habilitacion de muestreo (1 ms)
           boton    : in  STD_LOGIC;
           pulso    : out STD_LOGIC);
end antirrebote;

architecture Behavioral of antirrebote is

signal b1, b2   : STD_LOGIC;
signal muestras : STD_LOGIC_VECTOR (7 downto 0);
signal estable, estable_ant : STD_LOGIC;

begin

process (clk, rst)
begin
if rst = '1' then
    b1 <= '0';  b2 <= '0';
    muestras    <= (others => '0');
    estable     <= '0';
    estable_ant <= '0';
elsif clk'event and clk = '1' then
    b1 <= boton;
    b2 <= b1;
    if tick = '1' then
        muestras <= muestras(6 downto 0) & b2;
    end if;
    if muestras = "11111111" then
        estable <= '1';
    elsif muestras = "00000000" then
        estable <= '0';
    end if;
    estable_ant <= estable;
end if;
end process;

pulso <= estable and not estable_ant;

end Behavioral;
```

src/sincronizador.vhd

```
----------------------------------------------------------------------------------
-- SINCRONIZADOR DE LOS SWITCHES dir_p (dos flip-flops por bit)
-- Evita la metaestabilidad al pasar los switches al dominio del reloj.
----------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sincronizador is
    Port ( clk, rst : in  STD_LOGIC;
           entrada  : in  STD_LOGIC_VECTOR (2 downto 0);
           salida   : out STD_LOGIC_VECTOR (2 downto 0));
end sincronizador;

architecture Behavioral of sincronizador is

signal s1, s2 : STD_LOGIC_VECTOR (2 downto 0);
```

```
17
18  begin
19
20  process (clk, rst)
21  begin
22  if rst = '1' then
23      s1 <= "000";
24      s2 <= "000";
25  elsif clk'event and clk = '1' then
26      s1 <= entrada;
27      s2 <= s1;
28  end if;
29  end process;
30
31  salida <= s2;
32
33  end Behavioral;
```

src/display_8dig.vhd

```
 1  ----------------------------------------------------------------------------------
 2  -- DISPLAY DE 8 DIGITOS MULTIPLEXADO (Nexys A7: anodo comun, activos en bajo)
 3  --
 4  -- AN7 = signo ('-' o apagado); AN6..AN0 = 7 digitos BCD.
 5  -- Cambia de digito con cada tick (1 ms): barrido completo de 8 ms (125 Hz).
 6  ----------------------------------------------------------------------------------
 7  library IEEE;
 8  use IEEE.STD_LOGIC_1164.ALL;
 9  use ieee.std_logic_unsigned.all;
10
11  entity display_8dig is
12      Port ( clk, rst : in  STD_LOGIC;
13             tick     : in  STD_LOGIC;
14             bcd      : in  STD_LOGIC_VECTOR (27 downto 0);
15             neg      : in  STD_LOGIC;
16             an       : out STD_LOGIC_VECTOR (7 downto 0);
17             seg      : out STD_LOGIC_VECTOR (6 downto 0);   -- seg(0)=CA ... seg(6)=CG
18             dp       : out STD_LOGIC);
19  end display_8dig;
20
21  architecture Behavioral of display_8dig is
22
23  signal dsel      : STD_LOGIC_VECTOR (2 downto 0);   -- digito encendido
24  signal car       : STD_LOGIC_VECTOR (3 downto 0);   -- 0-9, A = '-', F = apagado
25  signal car_signo : STD_LOGIC_VECTOR (3 downto 0);
26
27  begin
28
29  process (clk, rst)
30  begin
31  if rst = '1' then
32      dsel <= "000";
33  elsif clk'event and clk = '1' then
34      if tick = '1' then
35          dsel <= dsel + '1';
36      end if;
37  end if;
38  end process;
39
40  car_signo <= "1010" when neg = '1' else "1111";
41
42  with dsel select
43      car <= bcd(3 downto 0)   when "000",
44             bcd(7 downto 4)   when "001",
45             bcd(11 downto 8)  when "010",
46             bcd(15 downto 12) when "011",
47             bcd(19 downto 16) when "100",
48             bcd(23 downto 20) when "101",
49             bcd(27 downto 24) when "110",
50             car_signo         when others;
51
52  with dsel select
53      an <= "11111110" when "000",
54            "11111101" when "001",
55            "11111011" when "010",
56            "11110111" when "011",
57            "11101111" when "100",
58            "11011111" when "101",
59            "10111111" when "110",
60            "01111111" when others;
61
62  with car select                                  --  gfedcba
63      seg <= "1000000" when "0000",                --  0
64             "1111001" when "0001",                --  1
65             "0100100" when "0010",                --  2
66             "0110000" when "0011",                --  3
67             "0011001" when "0100",                --  4
```

```
            "0010010" when "0101",              --  5
            "0000010" when "0110",              --  6
            "1111000" when "0111",              --  7
            "0000000" when "1000",              --  8
            "0010000" when "1001",              --  9
            "0111111" when "1010",              --  -
            "1111111" when others;              --  apagado

dp <= '1';

end Behavioral;
```